KH정보교육원

프로그래밍 언어 활용

JAVA

목차

Part 1. 구조적 프로그래밍 언어 활용하기

Chapter 1. 프로그래밍과 프로그램 언어 4

Chapter 2. 자바 소개 및 특징 6

Chapter 3. 자바 프로그래밍 기초 8

Chapter 4. 변수와 자료형 13

Chapter 5. 연산자 19

Chapter 6. 제어문 27

Chapter 7. 기본자료형 배열 38

Part 2. 객체지향 프로그래밍 언어 활용하기

Chapter 8. 클래스와 객체 45

Chapter 9. 객체배열 58

Chapter 10. 상속 61

Chapter 11. 다형성 69

Chapter 12. 공공데이터 객체와 연동하기 80

Part 1. 기본문법 활용하기

Chapter 1. 프로그래밍과 프로그래밍 언어

1 프로그래밍이란

1.1 프로그래밍 관련 용어

- **프로그램** : 컴퓨터를 작동시키기 위한 순차적으로 작성된 일련의 명령어들의 모음..
- **프로그래밍** : 프로그램을 만드는 행위. 프로그램 개발을 의미
- **프로그래머 :** 프로그램을 만드는 사람.. 프로그램 개발자를 의미
- **프로그래밍 언어** : 프로그램을 작성하기 위해 사용되는 언어

  프로그래밍 언어에 따라 같은 명령에 대해서도 명령을 내릴 때 사용하는 단어나 명령 체계가 언어마다 다르다. 사람이 이해하고 작성하기 쉬운 사람 중심 언어를 고급 언어라고 하고, 하드웨어 지향의 기계 중심 언어를 저급 언어라고 한다.
- **소프트웨어 (SoftWare : SW) :** 컴퓨터를 효율적으로 운영하기 위해 개발된 프로그램을 모두 소프트웨어라고 한다. 컴퓨터를 관리하는 시스템 소프트웨어와 문제 해결에 이용되는 다양한 형태의 응용 소프트웨어가 있다.

2 프로그램 언어별 특성

2.1 사용자 측면에서의 분류

프로그래밍 언어를 사용하는 개발자 측면에서 보는 관점에서 분류함.

특징	저급언어 Low Level Language	고급언어 High Level Language
주체	기계 중심의 언어	사용자 중심의 언어, 컴파일 언어
호환성	타 기계와 호환성 낮음	호환성 높음
에러수정	어려움	비교적 쉬움
프로그래밍 용이성	어려움	비교적 쉬움
수행속도	기계 자체 언어이므로 번역의 필요성 없어서 빠름	실행하기 위해 기계어로 번역해야 하므로 느림
대표언어	기계어, 어셈블리어	C, C++, JAVA, VISUAL C++, C#

2.2 실행 측면에서의 분류

프로그램을 실행하는 주체에 따라 나누는 방법으로 컴퓨터가 직접 실행하느냐 또는 매개체를 이용하느냐 등에 따라 구분.

- **컴파일러 언어 (Compiler Language)** : 완성된 소스 코드 자체로는 실행될 수 없고, 컴파일러(번역기)를 이용해 컴파일한 후에 만들어진 번역된 소스 코드가 실제 실행 프로그램이 된다. 일반적으로 고수준 언어에 의해 작성된 소스 코드가 실행될 컴퓨터 운영체제에 대한 기계언어로 번역된다.

- **인터프리터 언어 (Interpreter Language)** : 컴퓨터에 상주해 있는 인터프리터(실행기)가 소스 코드를 한 문장씩 읽어서 바로 실행한다.. 이러한 인터프리터를 이용한 개발 언어를 인터프리터 언어라고 한다.

- **스크립트 언어 (Script Language)** : 초창기에는 배치언어(batch language) 또는 작업 제어 언어(job control language) 라고 불리었으며, 응용 프로그램과 분리하여 작성하며 프로그램 사용자가 응용 프로그램의 동작을 요구에 맞게 작동될 수 있도록 해 주기 위한 목적으로 제공되는 스크립트 파일 작성용 언어이다.

언어	장점	단점	종류
컴파일러 언어	한번 컴파일하면 컴파일된 프로그램을 그대로 사용 가능하며 처리 시간이 매우 빠름.	한 줄의 소스 코드가 많은 기계어로 번역되기 때문에 상대적으로 큰 기억용량(Stack)이 필요함.	- C - C++ - Java - C#
인터프리터 언어	한 줄씩 해석하여 실행하기 때문에 기억장소가 많이 필요하지 않아 자원 효율적임. 플랫폼에 비의존적이고 자료형과 범위가 동적으로 설정될 수 있어 유연함.	인터프리터에 의해 해석되면서 실행되기 때문에 처리에 많은 시간이 소요되어 컴파일러 언어에 비해 비효율적임.	- Basic - Lisp - 포스트스크립트
스크립트 언어	특정 실행 환경상에서 실행되기 때문에 플랫폼 독립적임. 고수준언어로 프로그램 작성이 직관적으로 이루어질 수 있음.	단독으로 실행될 수 없기 때문에 별도의 런타임 환경을 구축해야 하고 경우에 따라 많은 리소스가 필요할 수 있음.	- JavaScript - ActionScript - AutoHotKey - Perl - Python - Ruby - VBS

Chapter 2. 자바 소개 및 특징

1 자바 소개

1.1 Java란

썬 마이크로시스템즈에서 개발하여 발표한 객체지향 언어로 운영체제에 영향을 받지 않고 실행되는 특징때문에 다양한 기종의 컴퓨터와 운영체제가 공존하는 인터넷 환경에 적합한 언어로 현재 많이 사용되고 있는 언어이다.

자바는 다양한 클래스 라이브러리(API)를 제공하고 있어 제공하는 클래스만 이용하여도 수준급의 프로그램을 만들 수 있다.

1.2 Java의 역사

Sun의 엔지니어들이 가전제품에 탑재할 소프트웨어를 만드는 것을 목표로 C++를 확장하여 사용할려고 했지만 c++로는 한계가 있다는 것을 깨닫고 c++을 장점을 도입하고 단점을 보완하여 Oak라는 새로운 언어를 개발하였다. 초기에는 가전제품이나 PDA와 같은 소형기기에 사용될 목적이였으나 인터넷의 등장으로 인터넷 환경에 맞게 개발방향을 변경하고 이름도 자바(Java)로 변경하였다. 그리고 현재는 웹개발에 많이 활용되고 있다.

2 자바특징

2.1 Java의 특징

- 객체 지향 언어이다.
- 플랫폼에 독립적이므로 이식성이 높다..
- 메모리를 자동으로 관리한다. (GC, Garbage Collection)
- 동적 로딩(Dynamic Loading)을 지원한다.
- 멀티 스래드를 쉽게 구현할 수 있다.
- 오픈 소스 라이브러리가 풍부하다.

2.2 Java의 플랫폼

- Java SE(Standard Edition) : 자바 기본 플랫폼
- Java EE(Enterprise Edition) : SE + WEB
- Java ME(Micro Edition) : 임베디드 플랫폼

2.3 Java의 프로그래밍 과정

Java Source Code –> Java Compiler -> Java Byte Code -> JVM 에 의해 재컴파일되어 실행
(파일명.java) (파일명.class)

① **.java 언어의 문법에 따라 소스 코드 작성한다. (클래스명.java)**

② **Jdk(Java Development Kit) 이 제공하는 javac 컴파일러를 사용하여 바이트코드 형식으로 소스코드를 컴파일 한다. (클래스명.class)**

③ **컴파일된 class 파일을 jdk가 제공하는 java 인터프리터를 사용하여 실행한다.**

- 실행시 실행에 필요한 클래스들이 JVM(Java Virtual Machine)에 연결되며 클래스 로더(Loader)가 필요한 클래스를 동적으로 로딩(Loading)한다

2.4 JVM(Java Virtual Machine)

- 자바 바이트코드를 해당 운영체제(Operation System) 기계어로 재번역하면서 실행하는 주체
- 운영체제와 자바 프로그램을 연결시켜주는 역할
- 자바 프로그램을 플랫폼(운영체제)에 독립적으로 동작할 수 있도록 해 주는 역할
- 인터프리터나 JIT 컴파일 방식으로 바이트코드를 실행할 수 있도록 함
- JRE에 포함되어 배포됨
- 가비지 컬렉션을 수행함
- 과거 순수 인터프리터 방식으로 실행시간이 늦은 편이었으나 JIT(Just-In-Time) 컴파일을 구현하여 성능을 개선하였음

*** 컴파일러 (Compiler)**

고급 언어로 쓰여진 프로그램이 컴퓨터가 이해할 수 있는 저급 언어로 번역
번역과정이 번거롭고 번역 시간이 오래 걸림
한번 번역한 후에는 다시 번역하지 않으므로 실행 속도가 빠름

*** 인터프리터 (Interpreter)**

프로그램을 한 단계씩 기계어로 해석하여 실행
실행 시간이 길어 속도가 늦음
프로그램이 직접 실행되므로 목적 프로그램이 생성되지 않음

*** JIT 컴파일**

프로그램을 실제 실행하는 시점에 기계어로 번역하는 컴파일 기법
실행 시점에 기계어 코드를 생성하면서 해당 코드를 캐싱
함수가 여러 번 호출될 때마다 매번 기계어 코드가 생성되는 걸 방지
인터프리터의 실행 속도가 느린 단점을 보완

Chapter 3. 자바 프로그래밍 기초

1 Java 소스코드 기본구조

자바 소스코드는 아래와 같은 구조로 되어있다.

```
01  package 패키지명; // 유사한 클래스들을 묶어줄 패키지 (폴더와 비슷) 선언
02
03  [ import 다른 패키지에 있는 클래스명; ] //외부클래스 사용 선언
04
05
06  class 클래스명 {
07      자료형 변수명 ; //멤버변수
08      public static void main(String[] args) { //main메소드
09
10                  시작될 기능 구현 코드
11
12      }
13
14      public 반환형 메소드명(매개변수, 매개변수, ....){
15
16                  매소드 기능구현 코드
17
18      }
19
20  }
```

☞ [ ] 표시는 생략이 가능

자바로 프로그래밍을 할때에는 위에 예제와 같은 구조로 작성을 해야한다. 먼저 클래스들을 묶어주는 패키지를 설정하고, 작성하고 있는 클래스(객체)에서 다른 패키지에 속해있는 클래스를 사용할 경우 import로 설정해주면 된다. 두가지를 설정하고 객체지향언어에서 제일 중요한 class를 설정하게 된다.
class내부에는 멤버변수와 메소드가 들어가게 되는데 이는 객체부분에서 설명하니 자세한 내용은 생략한다.

2 Java 프로그램의 실행순서

프로그램의 실행 순서는 코드의 윗쪽에서부터 아래로 차례대로 실행이되고, 특정한 메소드를 사용하지 않으면 다시 위로 윗쪽으로 올라가는 경우는 없다. 소스코드의 순서에 따라 영향을 받으니 초보들은 작성순서를 주의해서 작성해야 한다.

RunOrder.java

```
01  public class RunOrder {
02      public static void main(String[] args) {
03          System.out.println("Hello");
04          System.out.pritnln("java");
05      System.out.println("Welcome to Java World");
06      }
07  }
```

```
Hello
Java
Welcome to Java World
```

위의 예제는 Hello, Java, Welcome to Java를 출력하는 코드이다. 03번 소스부터 위쪽에서부터 순서대로 출력이 되는걸 볼 수가 있다. 이렇게 프로그램이 실행이 되면 소스코드를 위해서부터 아래로 읽게 된다. 다시 04번을 출력하고 03번을 출력하고 05번을 출력할 수는 없다.
실행순서는 명심해야한다. 순서(위치)를 잘못 지정하여 에러를 발생하는 경우가 많기 때문이다.
이번 예제는 순서를 잘못 지정하여 에러를 발생하는 코드이다.

RunOrderErr.java

```
01  public class RunOrderErr {
02      public static void main(String[] args) {
03          System.out.println(name);
04          String name="유병승";
05      }
06  }
07
```

```
에러발생!
```

이번 예제를 실행하면 에러가 name이라는 것이 없다고 에러를 발생시킬 것이다. 아직 변수나 기본문법을 배우지않았지만 순서를 설명하기 위해 작성하였다. 위에 04번 줄에서 name에 값을 넣고 03번 줄에서 출력하고 있다. 프로그램은 위에서 아래로 차례로 진행된다고 하였다. 그렇기 때문에 03번에서 출력하려고 할 때 그 위소스에서 name이라는 것이 없기 때문에 name이라는 것이 없다고 에러를 발생하는 것이다. 이것을 정상적으로 출력하려면 03번줄과 04번줄의 위치를 바꾸면 정상적으로 실행이 된다. 처음에는 이런 실수를 많이 하니 반드시 순서를 잘 기억해야 한다.

3 Java 프로그램 실행범위

실행범위는 코드상에서 '{ }' 로 표시하고 ' { ' 코드의 시작을 의미하고 ' } '는 코드의 끝을 의미 한다. 실행범위는 메소드, 클래스, 조건문, 반복문 등의 실행범위를 정할 때 사용한다. 앞에 나온 용어는 앞르로 배울것이니 이런 것이 있구나 하고 넘어가도록 한다. 실행범위 내에 있는 소스들 즉 " **{** "와 " **}** "사이에 있는 소스코드만 사용이 가능하고 다른 실행범위에 있는 소스코드는 일반적으로 사용할 수 없다. 실행범위설정에서도 많이 실수를 하니 잘 기억해야 한다.

RunScope.java

```
01  public class RunScope {
02      public static void main(String[] args) { //메인메소드
03          String name="홍길동";
04          System.out.println(name);
05          print();
06      }
07      public static void print() { //print메소드
08          System.out.println(name);
09      }
10  }
```

에러발생

위 예제가 발생하는 이유는 name은 main()메소드(02번줄) 실행범위 안에 있기 때문에 print()에서는 사용할 수가 없기 때문에 에러가 발생한다. 예제를 아래와 같이 수정하면 에러발생 없이 실행이 되고 똑같은 name이라는 것을 출력하지만 각각 다른 내용을 출력하게 되는데 각자 실행범위에 있는 것을 불러오기 때문이다.

RunScope1.java

```
01  public class RunScope1 {
02      public static void main(String[] args) { //메인메소드
03          String name="유병승";
04          System.out.println(name);
05          print();
06      }
07      public static void print() { //print메소드
08          String name="김자바"
09          System.out.println(name);
10      }
    }
```

유병승
김자바

4 주석 (Comment)

소스 코드의 내용 이해를 돕기 위한 설명 문구를 주석이라고 한다.

한 줄 주석 : // 소스 코드 내용 이해를 돕기 위한 설명 문구를 한 줄로 기록

여러 줄 주석 : /* 소스 코드 내용 이해를 돕기 위한 설명 문구를 여러 줄로 기록 */

Java document 용 주석 : /** 작성된 클래스에 대한 API 도큐먼트 문서에 기록될 내용 입력 */

Comment.java

```
01  public class Comment {
02      public static void main(String[] args) {
03          // 한 줄 주석
04          System.out.println("Hello");
05          // System.out.println("Java"); // 이 주석 기호의 오른쪽은 코드로 인식되지 않습니다
06
07          /*
08          여러 줄 주석
09          이 안의 내용은 코드로 인식되지 않습니다.
10          컴파일시 소스 코드에서 제외되고 컴파일됩니다.
11          System.out.println("Hello Java");
12          */
13      }
    }
```

Hello

Java document 만들기

TestDoc.java

```
01  /**
02      자바 클래스에 대한 API 도큐먼트 작성 테스트용 클래스이다.
03  */
04  public class TestDoc {
05  /**
06     이름 저장용 멤버변수
07  */
08      private String name;
09
10  /**
11      나이 저장용 멤버변수
12  */
```

```
13      private int age;
14
15  /**
16     객체 생성시 멤버변수 초기화용 매개변수 있는 생성자
17  */
18      public TestDoc(String name, int age) {
19          this.name = name;
20          this.age = age;
21       }
22
23  /**
24      멤버변수가 가진 값을 콘솔 화면에 출력시키는 메소드
25  */
26       public void print( ) {
27          System.out.println(name + ", " + age);
28     }
29  }
```

이클립스 상단 메뉴바에서

project > Generate javadoc... 선택

> javadoc Generantion 창에서 'javadoc command' 에 설치된 jdk 폴더 아래 bin 폴더 아래의 javadoc 실행파일을 선택한다.

> 도큐먼트 문서 만들 클래스를 체크하고, 생성될 위치 확인하고

> Finish 버튼을 누른다.

> 생성된 doc 폴더 안의 index.html 을 실행시켜 확인해 본다.

Chapter 4. 변수와 자료형

1 변수

1.1 변수의 정의

- 메모리에 하나의 값(data)을 저장할 수 있는 방과 같은 기억 공간(메모리 공간)이다.
- 변수 공간에 기록된 값(data)은 고정되어 있지 않고 다른 값(data)이 기록이 되면 자동 덮어쓰기되어 변할 수 있다.
- 기록 저장되는 값(data)의 종류에 따라 변수 공간의 자료형(Data Type)이 결정된다.

2 자료형 (Data Type)

2.1 변수 공간에 기록되는 값의 종류

2.1.1 기본형(Primitive Data Type)

- 자바는 값(data)을 8개의 기본형으로 구분 (boolean, char, byte, short, int, long, float, double)
- Java 컴파일러에 의해서 해석되는 자료형

2.1.2 기본 자료형의 종류

자료형	키워드	크기	기본값	값의 범위
논리형	boolean	1byte	false	true 또는 false
문자형	char	2byte	'₩u0000'	0 ~ 65,535
정수형	byte	1byte	0	-128 ~ 128
	short	2byte	0	-32,768 ~ 32,767
	int	4byte	0	-2^{31} ~ $2^{31}-1$
	long	8byte	0L	-2^{63} ~ $2^{63}-1$
실수형	float	4byte	0.0f	-3.4E38 ~ +3.4E38
	double	8byte	0.0 또는 0.0d	-1.7E308 ~ +1.7E308

2.1.3 참조형(Reference Data Type)

- 클래스를 사용자 정의 자료형이라고 하며, 클래스를 자료형으로 하여 선언된 변수를 레퍼런스(Reference) 변수라고 한다.
- 자유메모리 영역인 Heap 메모리에 생성된 객체의 주소를 저장한다.
- 자바에서는 메모리 주소를 직접 소스코드로 참조하는 것은 불가능하다.

3 변수의 사용

3.1 변수 선언

자료형 변수명;

TestVariable.java

```
01 public class TestVariable {
02     public static void main(String[] args) {
03         boolean flag;
04         char grade;
05         byte value;
06         short svalue;
07         int num;
08         long price;
09         float tax;
10         double score;
11     }
12 }
```

3.2 변수 초기화

변수 선언과 동시에 초기값 대입 처리하는 것을 말한다.

자료형 변수명 = 초기값;

TestVariableInit.java

```
01 public class TestVariableInit {
02     public static void main(String[] args) {
03         boolean flag = false;
04         char grade = 'A';
05         byte value = 90;
06         short svalue = 2500;
07         int num = 1250000;
08         long price = 2350L;
09         float tax = 3.3f;
10         double score = 99.5;
11
12         System.out.println("변수 공간에 기록된 초기값 확인");
13         System.out.println("flag : " + flag);
14         System.out.println("grade : " + grade);
```

```
15          System.out.println("value : " + value);
16          System.out.println("svalue : " + svalue);
17          System.out.println("num : " + num);
18          System.out.println("price : " + price);
19          System.out.println("tax : " + tax);
20          System.out.println("score : " + score);
21      }
22  }
```

3.3 변수 활용

소스 코드상에서 변수명의 사용은 변수 공간에 기록된 값의 사용을 뜻한다.

TestUsingVariable.java

```
01  public class TestUsingVariable {
02      public static void main(String[] args) {
03          int price = 2500, quantity = 3;
04          int totalPrice = price * quantity;
05
06          System.out.println("가격 : " + price + " 원, 수량 : " + quantity + "개");
07          System.out.println("총 구매가격 : " + totalPrice + " 원");
08      }
09  }
```

3.4 변수 선언시 변수명 작성 규칙

- 대소문자가 구분되며 길이에 제한이 없다.
- 예약어(Keyword)를 사용할 수 없다.
- 숫자로 시작해서는 안 된다.
- 특수문자는 '_'와 '$'만 허용된다.

3.5 일반적인 관례

- 클래스 이름은 반드시 대문자로 시작한다. (Product, Manager, Member 등)
- 메소드(멤버함수) 이름, 변수명은 소문자로 시작하며, 여러 단어를 합쳐서 의미가 부여된 이름으로 작성한다. (printMember, calculatorScore, setName, getAge 등)
- 상수변수 이름은 모두 대문자로 작성한다. (PI, SIZE 등)
- 여러 개의 단어가 이어져서 작성된 이름일 경우에는 낙타봉 표기법을 사용하여, 두번째 단어이후의 단어들은 첫 글자를 대문자로 한다. (toUpperCase, equalsEgnoreCase 등)
- 상수변수 이름이 여러 개의 단어로 구성될 경우에는 단어간 구분에는 '_' (under score)를 이용한다. (MAX_SIZE)

4 예약어

- 해당 프로그램 언어의 필요에 의해서 기능이 이미 정해져 있는 단어들 (Keyword)
- 자바 예약어는 모두 소문자로 대략 50여 개로 이루어져 있다.
- 예약어는 변수명, 클래스명, 메소드명으로 사용할 수 없다.
- 예약어 종류

abstract	assert	boolean	break	byte
case	catch	char	class	const
continue	default	do	double	else
enum	extends	false	final	finally
float	for	goto	if	implements
import	instanceof	int	interface	long
native	new	null	package	private
protected	public	return	short	static
strictfp	super	switch	synchronized	this
try	void	while		

5 형변환(Type Casting)

- 데이터의 자료형을 다른 자료형으로 바꾸는 것
- 컴퓨터에서 데이터는 서로 같은 자료형일 때 연산/대입이 가능하다.
- 서로 다른 자료형 데이터들의 연산과 대입시에는 같은 자료형으로 변경시킨 후에 연산과 대입이 가능하다
- 기본 자료형과 참조 자료형 모두 형변환이 가능하다.
- boolean은 false(bit 0)와 true(bit 1)만을 저장하기 위해 특별히 만들어진 데이터 타입이므로 다른 자료형과의 형 변환이 불가능하다.
- 형변환에는 명시적(강제) 형변환과 묵시적(자동) 형변환으로 구분된다.

5.1 묵시적(자동) 형변환 (Promotion)

- 프로그램 실행 도중 자동으로 일어나는 형변환
- 메모리 할당 크기가 작은 자료형 데이터를 메모리 할당 크기가 큰 자료형 변수에 대입하는 경우 자동으로 형변환이 발생한다.
- 자료형이 다른 데이터들이 섞여서 연산이 될 경우에도 자동 형변환이 발생한다.

* 예외

char 자료형은 메모리 할당 크기가 2byte이고 값의 표현범위가 0~65,535이며 음수를 저장하지 않는다. 그러므로 음수가 저장될 수 있는 정수의 byte 자료형이나 short 자료형은 char타입으로 자동 형변환할 수 없다. 자바에서 char 자료형과 형변환이 가능한 정수 자료형은 int 이다.

TestPromotion.java

```
01  public class TestPromotion {
02      public static void main(String[] args) {
03          int intVal = 100;
04          double doubleVal = intVal;  // 100 이 100.0 으로 자동 형변환된 다음 대입됨
05          System.out.println("intVal : " + intVal + ", doubleVal : " + doubleVal);
06
07          char chVal = 'A';
08          int uniCode = chVal;    //문자 'A' 의 유니코드 65, 즉 정수로 자동 형변환된 다음 대입됨
09          System.out.println("chVal : " + chVal + ", uniCode : " + uniCode);
10      }
11  }
```

5.2 명시적(강제) 형변환 (Casting)

(바꾸고자 원하는 자료형)데이터

(바꾸고자 원하는 자료형)변수

- 메모리 할당 크기가 큰 자료형 데이터를 크기가 작은 자료형으로 바꾸려면 값의 손실이 발생하는 경우이므로 코드상에 직접 형변환을 명시해야 한다.
- 같은 자료형 데이터끼리 연산하더라도 결과값이 해당 자료형 값의 범위를 초과하는 경우가 발생할 경우, 결과값에 대한 자료형으로 연산 값들의 자료형을 변경해야 하는 경우에도 형변환을 직접 명시한다.

TestCasting.java

```
01  public class TestCasting {
02      public static void main(String[] args) {
03          long longVal = 100L;
04          int intVal = (int)longVal;  // 8바이트짜리 100이 4바이트짜리 100으로 변환됨
05          System.out.println(intVal);
06
07          double dValue = 123.5;
08          int iValue = (int)dValue;  //8바이트 실수형 값이 4바이트 정수형 값으로 변환됨
```

```
09          System.out.println("dValue : " + dValue + ", iValue : " + iValue);
10
11          int maxValue = Integer.MAX_VALUE;
12          long result = (long)maxValue * maxValue;   //형변환하지 않으면 오류 발생함
13          System.out.println("result : " + result);
14      }
15  }
```

Chapter 5. 연산자

1 연산자 (Operator)

값 계산에 사용되는 계산용 기호 문자

- **피연산자(Operand) :** 연산의 대상이 되는 데이터(값)

1.1 연산자의 종류

- 연산자를 분류하는 방법에는 기능에 따른 분류, 피연산자 개수에 따른 분류가 있다.

1.1.1 기능에 따른 분류

종류	연산자	기능
산술 연산자	+ - * / % << >> >>>	사칙연산을 비롯한 산술적인 연산
관계 연산자	< > <= >= == !=	피연산자들의 크고 작은 관계를 따지는 연산
논리 연산자	&& || ! & | ^ ~	AND와 OR 등 참, 거짓으로 나뉘는 논리적 연산
대입 연산자	=	우변의 값을 좌변에 저장하는 연산
기타	(type) ?: instanceof	형변환, 삼항조건, 객체타입검사

1.1.2 피연산자 개수에 따른 분류

연산에 필요한 피연산자의 개수에 따라 분류하는 방법이다.

'-' 연산자는 연산에서 두 가지 역할을 한다. 피연산자가 음수임을 나타내는 단항 연산자와 두 피연산자의 뺄셈 연산으로 사용하는 이항연산자가 있다..

연산에 필요한 항의 개수별로 연산자를 나누면 단항연산자, 이항연산자, 삼항연산자로 나눌 수 있다.

1.2 연산자 우선순위

1.2.1 연산자 결합규칙

컴퓨터는 연산을 한번에 한 개만 수행할 수 있고 연산자는 연산을 수행하고 나면 항상 결과값을 반환한다. '3 + 5 + 7' 같은 연산식은 먼저 '3 + 5' 연산이 수행되고 '8'을 반환하고 '8 + 7'이 연산되어 최종 결과인 '15'를 반환한다. '3 + 5 + 7' 같은 연산의 경우 왼쪽에서부터 오른쪽으로 연산이 진행됐지만 연산자들은 우선순위에 따라 한 연산식 내에서 연산 순서가 바뀔 수 있다.

1.2.2 연산자 우선순위

우선 순위	연산자	연산 방향
높음 ↑	최우선 : ( ), [ ], ., ->	
	단항 : 증감(++, --), 부호(+, -), 비트반전(~), 논리부정(!), (자료형)	←
	이항 : 산술(*, /, %)	→
	이항 : 산술(+, -)	→
	이항 : 시프트(<<, >>, >>>)	→
	이항 : 관계(<, >, <=, >=, instanceof)	→
	이항 : 관계(==, !=)	→
	이항 : 비트논리 AND(&)	→
	이항 : 비트논리 XOR(^)	→
	이항 : 비트논리 OR(|)	→
	이항 : 일반논리(&&)	→
	이항 : 일반논리(||)	→
	삼항(? :)	→
낮음 ↓	대입(=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=)	←

1.3 단항 연산자

1.3.1 부호(Sign) 연산자 +, -

부호 연산자 '+'는 (값 * 1)의 의미를 가지는 연산자로 피연산자의 부호를 그대로 유지한다. 반대로 '-' 부호연산자는 (값 * -1)의 의미를 가지는 연산자로 피연산자의 부호를 반대로 바꾸는 연산자이다.

TestSign.java

```
01  public class TestSign{
02      public static void main(String[] args) {
03    int value = -57;
04
05    int result1 = +value;
06    int result2 = -value;
07
08              System.out.println("value : " + value);
09    System.out.println("result1 = " + result1);
10    System.out.println("result2 = " + result2);
11      }
12  }
```

```
value : -57
result1 : -57
result2 : 57
```

1.3.2 증감 연산자 ++, --

피연산자에 저장된 값을 1 증가 또는 1 감소시킨다.

```
++ : 1 증가연산자
-- : 1 감소연산자
```

증감연산자는 피연산자의 왼쪽 또는 오른쪽 양쪽에 다 위치할 수 있다. 이 때 연산자가 '++num' 과 같이 왼쪽에 있으면 전위증감연산자라 하고, 'num++' 과 같이 오른쪽에 있으면 후위증감연산자라고 한다..
피연산자와 단독으로 증감연산자가 사용될 때에는 전위든 후위든 같은 결과를 가지지만, 다른 연산과 결합해서 일부로 사용될 때에는 전위와 후위의 결과과 달라진다. 전위로 사용된 증감연산자는 변수의 값을 먼저 증감시킨 후 결과를 가지고 연산하고 후위로 사용된 증감연산자는 변수의 값을 연산에 사용한 후에 변수의 값을 증감시킨다.

TestIncDec.java

```
01  public class TestIncDec {
02      public static void main(String[] args) {
03    int x = 12;
04    int y = 12;
05
06    int result1 = ++x + 10;
07    int result2 = y++ + 10;
08
09    System.out.println("x : " + x + ", result1 = " + result1);
10    System.out.println("y : " + y + ", result2 = " + result2);
11              System.out.println("y : " + y);
12      }
13  }
```

```
x : 13, result1 = 23
y : 12, result2 = 22
y : 13
```

1.3.3 논리 부정 연산자 !

논리값 true / false 를 반대로 바꾼다. 피연산자가 true이면 false를, false이면 true를 반환한다.

1.3.4 비트 반전 연산자 ~(Tield)

정수에만 사용되며 피연산자의 이진수 비트값 0은 1로, 1은 0으로 바꾸는 연산자

부호 있는 정수 타입을 연산할 경우 부호가 반대로 변경되며, 연산 결과로 해당 정수의 1의 보수를 얻을 수 있다.

TestTield.java

```
01  public class TestTield {
02      public static void main(String[] args) {
03   int data = 10;
04
05   System.out.println("data : " + data);
06   System.out.println("10의 1의 보수 : " + ~data);
07            System.out.println("10의 0의 보수 : " + (~data + 1));
08      }
09  }
```

```
data : 10
10의 1의 보수 : -11
10의 0의 보수 : -10
```

1.4 이항 연산자

반드시 두 개의 피연산자가 연산자 앞과 뒤에 있어야 한다.

1.4.1 산술 연산자 +, -, *, /, %

더하기(+), 빼기(-), 곱하기(*), 나누기한 몫(/), 나누기한 나머지(% : mod)를 연산할 때 사용한다.

TestArithmetic.java

```
01  public class TestArithmetic {
02      public static void main(String[] args) {
03   int x = 10;
04   int y = 7;
05
06   System.out.println(x + " + " + y + " = " + (x+y) );
07   System.out.println(x + " - " + y + " = " + (x-y) );
08   System.out.println(x + " * " + y + " = " + (x*y) );
09   System.out.println(x + " / " + y + " = " + (x/y) );
10   System.out.println(x + " / " + y + " = " + (x/y) );
11   System.out.println(x + " % " + y + " = " + (x%y) );
12      }
13  }
```

나누기 연산에서 두 피연산자가 모두 int 타입일 경우, 연산 결과의 데이터 타입은 int 형이다. 이 결과에서 소수점 이하 데이터는 버려지게 되고 결과로 나온 정수값은 나눗셈의 몫이 된다. 소수점 이하의 결과를 포함하는 나눗셈 결과를 얻고자 할 때에는 두 개의 피연산자 중 한 개라도 소수점을 포함할 수 있는 데이터 타입(double 또는 float)으로 형변환해야 한다. 또한 피연산자가 정수형일 경우 나누는 수로 0을 사용할 수 없다.

1.4.2 관계 연산자 <, >, <=, >=, ==, !=

피연산자들의 대/소/등/부등 관계를 따져 true / false로 결과를 반환한다.

- **대/소비교 < > <= >=**

두 피연산자의 크기를 비교하여 true나 false를 반환한다.

Boolean 형을 제외한 기본형 데이터에 사용하며, 참조형에는 사용할 수 없다.

관계 연산자	연산결과
좌변 < 우변	좌변이 작으면 true, 아니면 false (초과)
좌변 > 우변	좌변이 크면 true, 아니면 false (미만)
좌변 <= 우변	좌변이 작거나 같으면 true, 아니면 false (이상)
좌변 >= 우변	좌변이 크거나 같으면 true, 아니면 false (이하)

- **등가비교 ==, !=**

두 피연산자가 같은지 다른지 비교하여 true나 false를 반환한다.

기본형과 참조형, 모든 자료형에 사용할 수 있다. 기본형의 경우 변수에 저장되어 있는 값이 같은지 비교하며, 참조형의 경우 참조하고 있는 대상 객체가 같은지 비교한다.

관계 연산자	연산결과
좌변 == 우변	두 값이 같으면 true, 아니면 false
좌변 != 우변	두 값이 다르면 ture, 아니면 false

1.4.3 논리 연산자 &&, ||, !

&& (AND)와 || (OR) 연산자 두 가지가 있으며, '그리고, ~면서' 와 '또는, ~거나' 의 의미를 가지고 있다. && 연산은 두 피연산자가 모두 true일 경우 true를 반환하고, || 연산은 두 피연산자 중 한 개라도 true일 경우 true를 반환한다.

x	y	x && y	x || y
true	true	true	true
true	false	false	true
false	true	true	false
false	false	false	false

1.4.4 비트 논리 연산자 &, |, ^

정수에만 사용되며 피연산자의 자리별 비트값들을 각각 논리 연산으로 비교하여 정수 값을 결과로 반환한다.

- **비트 논리 연산자**

두 정수 피연산자들의 이진수 비트들을 각각 논리 연산하여 결과를 반환한다.

&(AND)	두 피연산자의 두 비트가 모두 1이면 1을 반환, 아니면 0을 반환
|(OR)	두 피연산자의 한 쪽 비트라도 1이면 1을 반환, 아니면 0을 반환
^(XOR)	두 피연산자의 비트가 다르면 1을 반환, 같으면 0을 반환

*** true / false 와 비트값과의 관계**

컴퓨터는 2진수를 다루는 기계이다. 2진수의 세계에는 0과 1밖에 존재하지 않으며 이 두 가지를 이용해 논리구조를 표현한다. 기본적으로 0은 false를 1은 true를 나타낸다.

1.4.5 시프트 연산자 <<, >>, >>>

비트 연산자에 해당되며 정수 피연산자의 비트들을 왼쪽(<<) 또는 오른쪽(>>, >>>)으로 이동시켜 정수값의 변화를 주는 연산자

- **왼쪽 시프트 연산자 <<**

정수 피연산자의 비트열 각 자리를 왼쪽으로 이동시킨다. 비트 이동후 비워지는 오른쪽은 양수/음수 상관없이 비트 0으로 채운다.

연산 결과는 비트 한 자리를 이동시킬 때마다 2의 배수를 곱하기한 결과와 같다.

- **부호있는 오른쪽 시프트 연산자 >>**

비트열을 오른쪽으로 이동시킨 후 왼쪽의 빈 공간에 양수는 0으로, 음수는 1로 채운다.

- **부호없는 오른쪽 시프트 연산자 >>>**

비트열을 오른쪽으로 이동시킨 후 왼쪽의 빈 공간에 양수/음수 상관없이 모두 0으로 채운다.

수식	자리이동	2진수	10진수
9<<0	없음	00000000 00000000 00000000 00001001	9
9<<1	왼쪽 한 번	00000000 00000000 00000000 00010010	18
-9<<2	왼쪽 두 번	00000000 00000000 00000000 00100100	36
-9<<0	없음	11111111 11111111 11111111 11110111	-9
-9<<1	왼쪽 한 번	11111111 11111111 11111111 11110111	-18
-9<<2	왼쪽 두 번	11111111 11111111 11111111 11110111	-36

9>>0	없음	00000000 00000000 00000000 00001001	9
9>>1	오른쪽 한 번	00000000 00000000 00000000 00000100	4
9>>2	오른쪽 두 번	00000000 00000000 00000000 00000010	2
-9>>0	없음	11111111 11111111 11111111 11110111	-9
-9>>1	오른쪽 한 번	111111111 11111111 11111111 1111011	-5
-9>>2	오른쪽 두 번	1111111111 11111111 11111111 111101	-3

9>>>0	없음	00000000 00000000 00000000 00001001	9
9>>>1	오른쪽 한 번	00000000 00000000 00000000 00000100	4
9>>>2	오른쪽 두 번	00000000 00000000 00000000 00000010	2
-9>>>0	없음	11111111 11111111 11111111 11110111	-9
-9>>>1	오른쪽 한 번	011111111 11111111 11111111 1111011	2147483643
-9>>>2	오른쪽 두 번	0011111111 11111111 11111111 111101	1073741821

1.5 대입 연산자 =

대입연산자 '='를 사용해서 변수 공간에 값을 저장할 수 있다.

```
num = 10;
```

1.5.1 복합 대입 연산자

+=, -=, *=, /=, %=, <<=, >>=, >>>=, &=, |=, ^=

대입 연산자와 다른 연산자를 결합하여 '연산자=' 방식으로 사용된다.

복합 대입 연산자	대입 연산자
i += 2;	i = i + 2;
i -= 2;	i = i - 2;
i *= 2;	i = i * 2;
i /= 2;	i = i / 2;
i %= 2;	i = i % 2;
i <<= 2;	i = i << 2;
i >>= 2;	i = i >> 2;
i >>>= 2;	i = i >>> 2;
i &= 2;	i = i & 2;
i |= 2;	i = i | 2;
i ^= 2;	i = i ^ 2;

1.6 삼항 연산자 ? :

삼항 연산자는 조건식을 가지고 반환할 피연산자를 선택하는 동작을 하기 때문에 조건 연산자라고도 부른다.

```
조건식 ? 값1 : 값2
```

조건식의 결과가 true일 경우에는 값1을 반환하고, false일 경우에는 값2를 반환한다. 조건식은 결과값이 반드시 boolean형이어야 한다.

TestTernary.java

```
01 public class TestTernary{
02     public static void main(String[] args) {
03   int value = 123;
04             String result = (value % 2 == 0)? "짝수": "홀수";
05   System.out.println( value + "는 " + result + "이다.");
06     }
07 }
```

```
123는 홀수이다.
```

Chapter 6. 제어문

1 제어문

프로그램의 실행 흐름을 제어하는 문장을 제어문이라고 한다. 프로그램의 실행 흐름은 메소드 블록 { } 안에서 작성한 순서대로 실행된다. 개발자가 원하는 방향으로 실행의 흐름을 바꾸는 것이 제어문이다. 제어문은 조건문, 반복문, 분기문으로 나뉜다.

1.1 조건문

조건식을 제시하고 조건식의 결과에 따라 다음 코드의 수행 여부를 결정하는 문장이다. 조건식은 반드시 true 또는 false 의 결과가 나오게 작성하여야 한다. 조건문에는 if문과 switch문이 있다.

1.1.1 if문

제시된 조건식의 결과가 true 일 때만 { } 안의 문장들을 실행시킨다.

```
if (조건식) {
   // 조건식이 참(true)일 경우 실행되는 문장들
}
```

- tru **조건식**

boolean 타입 (true / false)로 결과 값이 반환되는 식

일반적으로 e / false 의 결과를 만드는 관계연산자와 논리연산자가 주로 사용된다.

조건식	조건식이 참인 조건
x >= 1 && x <= 100	변수 x가 가진 값이 1부터 100사이의 값이냐?
ch == 'y' || ch == 'Y'	변수 ch가 가진 문자값이 소문자 'y' 또는 대문자 'Y' 이냐?
ch >= 'a' && ch <= 'z'	변수 ch가 가진 문자값이 영어 소문자이냐?

TestIf.java

```
01  public class TestIf {
02     public static void main(String[] args) {
03        int num = 100;
04
05        if(num > 0)      // num이 양수일 때
06           System.out.println("num은 양수이다");
07  }
```

1.1.2 if – else문

if 문의 조건식의 결과가 true 일 경우와 false일 경우 수행할 문장을 각각 지정한다. 제시된 조건식의 결과가 false 일 때를 의미하는 단어는 else 로 표현한다.

```
if (조건식) {
   // 조건식이 참(true)일 경우 실행되는 문장들
} else {
   // 조건식이 거짓(false)일 경우 실행되는 문장들
}
```

조건식의 결과가 true 이면 if문 블록만, 결과가 false이면 else문 블록만 수행한다.

TestIfElse.java

```
01 public class TestIfElse {
02    public static void main(String[] args) {
03       int num = 100;
04
05       if(num % 2 == 0) {
06          // num을 2로 나눈 나머지가 0 이냐? 즉 짝수냐?
07          System.out.println("num은 짝수이다");
08       } else {
09          // num 을 2로 나눈 나머기가 0이 아니냐? 즉 홀수냐?
10          System.out.println("num은 홀수이다");
11       }
12    }
13 }
```

1.1.3 if – else if문

다중 if 문이라고도 하며 판단해야 하는 조건이 세 가지 이상인 경우에 사용한다.
첫번째 if 문의 조건식을 판단하고 거짓이라면 다음 else if 문으로 넘어가서 계속해서 조건식을 판단한다. 조건식이 참(true)일 경우에는 다음으로 넘어가지 않고 해당 블록 안의 문장을 수행하고 구문이 종료되며 더 이상 다음 else if문을 수행하지 않는다.

```
if (조건식1) {
   // 조건식1이 참(true)일 경우 실행되는 문장들
} else if (조건식2) {
   // 조건식1이 거짓(false)이고, 조건식2가 참(true)일 경우 실행되는 문장들
} else if (조건식3) {
```

```
    // 조건식1, 2가 거짓(false)이고, 조건식3이 참(true)일 경우 실행되는 문장들
} else {
    // 조건식1, 2, 3 모두 거짓(false)일 경우 실행되는 문장들
}
```

TestMultilfElse.java

```
01  public class TestMultilfElse{
02      public static void main(String[] args) {
03          int score = 89;
04          char grade = ' ';
05
06          if(score >= 90) {           // 90점 이상일 경우 A학점
07              grade = 'A';
08          } else if(score >= 80) {    // 90점 미안 80점 이상일 경우 B학점
09              grade = 'B';
10          } else if(score >= 70) {    // 80점 미만 70점 이상일 경우 C학점
11              grade = 'C';
12          } else {                    // 70점 미만일 경우 F학점
13              grade = 'F';
14          }
15
16          System.out.println("점수 : " + score + ", 학점 : " + grade);
17      }
18  }
```

1.1.4 중첩 if 문

if 문의 블록 내에 다른 if 문을 포함시키는 구문이며, 중첩 횟수에는 제한이 없다.

```
if (조건식1) {
    // 조건식1이 참(true)일 경우 실행되는 문장들
    if (조건식2) {
        // 조건식1이 참(true)이고 조건식2가 참(true)일 경우 실행
    } else {
        // 조건식1이 참(true)이고 조건식2가 거짓(false)일 경우 실행
    }
} else {
    // 조건식1이 거짓(false)일 경우 실행되는 문장들
}
```

TestDupIf.java

```
01  public class TestDupIf{
02     public static void main(String[] args) {
03        int score = 89;
04        String grade = "";
05
06        if(score >= 90) {  // 90 점 이상일 경우 'A' 대입
07           grade = "A";
08           if(score >= 98) {  // 90 점 이상인 값 중 98점 이상일 경우 '+' 추가
09              grade += "+";
10           }
11        } else if(score >= 80) {  // 80 점 이상일 경우 'B' 대입
12           grade = "B";
13           if(score >= 88) {  // 80 점 이상인 값 중 88점 이상일 경우 '+' 추가
14              grade += "+";
15           }
16        } else if(score >= 70) {  // 70 점 이상일 경우 'C' 대입
17           grade = "C";
18           if(score >= 78) {  // 70 점 이상인 값 중 78점 이상일 경우 '+' 추가
19              grade += "+";
20           }
21        } else {     // 그 이하 점수일 경우 'F' 대입
22           grade = "F";
23        }
24
25        System.out.println("점수 : " + score + ", 학점 : " + grade);
26     }
27  }
```

1.1.5 switch문

switch문은 if 문과 다르게 값에 따라 준비된 구문이 수행되게 하는 구조를 가진 조건 선택문이다. if 문의 조건식과 다르게 값이 무엇인지를 묻는 계산식이나 변수를 제시하여 case 문의 값을 비교하여 실행할 문장을 선택 결정한다. case 문에 제시되는 값은 정수형 데이터, 문자형 데이터, 문자열 값만 사용된다.

```
switch (선택표현식) {
  case 값1 :
    // 조건식의 결과가 값1과 같을 경우 수행되는 문장들
    break;
  case 값2 :
    // 조건식의 결과가 값2과 같을 경우 수행되는 문장들
    break;
  case ... :
  default :
    // 조건식의 결과와 일치하는 case가 없을 경우 수행되는 문장들
}
```

TestSwitch.java

```
01  public class TestSwitch {
02     public static void main(String[] args) {
03           int score = 89;
04           char grade = ' ';
05
06           switch(score/10) {
07           case 10:
08           case 9:       grade = 'A';
09                         break;
10           case 8:      grade = 'B';
11                         break;
12           case 7:      grade = 'C';
13                         break;
14           default:    grade='F';
15        }
16
17        System.out.println("점수 : " + score + ",  학점 : " + grade);
18     }
19  }
```

switch 문에서의 break 문은 각 case 문의 영역을 구분하는 역할을 하며, break 문을 생략하면 case 문 사이의 구분이 없어지므로 다음 break 문을 만나거나 switch 문 블록 { } 의 끝을 만날 때까지 다른 case 에서 수행될 모든 문장을 수행하면서 내려간다. 이러한 fall throught 현상을 이용하여 일부러 break 문을 사용하지 않고 switch 문을 작성하는 경우도 있으나, 일반적으로 case 문에는 반드시 실행 구문 마지막에 break 문을 넣어주어야 한다.

1.2 반복문 (LOOP 문)

문장들을 반복해서 여러 번 수행되게 할 때 사용하는 구문이다. 구문 상에 반복되는 구간을 루프(LOOP) 라고 하며, 루프가 있는 문장이라고 하여 루프(LOOP) 문이라고도 한다.

1.2.1 for 문

가장 많이 사용되는 반복문으로 정확한 반복 횟수가 정해져 있을 때 주로 사용한다.
배열이나 컬렉션의 연속 데이터 처리에 주로 사용된다.
for 문은 초기식, 조건식, 증감식으로 구성되며, 조건식이 참(true) 일 동안 계속 반복되는 구조로 작동되는 구문이다.

```
for (초기식; 조건식; 증감식) {
  // 반복 수행될 코드 작성 영역
  // 조건식이 참(true)일 경우 수행되는 문장들
}
```

for문에서 초기식은 반복하는 횟수 카운트용 변수 선언과 동시에 카운트 초기값을 지정하는 부분으로 딱 한 번 실행된다.
반복 조건식은 반복할 카운트의 종료값을 제시하며 조건식이 참(true) 이면 아래의 블록 { } 안의 내용이 수행되며, 조건식이 거짓(false)이면 반복을 종료한다.
증감식은 블록 { } 안의 내용이 수행이 끝나면 카운트용 변수의 값에 증감 연산이 수행되고, 다시 앞의 조건식이 수행된다.
for문은 항상 조건식을 판별한 후에 끝난다. 즉 for 문은 초기식 –> 조건식 –> 수행코드 –> 증감식 -> 조건식 -> 수행코드 -> 증감식 의 순서로 실행되며, 증감식의 값의 증감량에 따라 for문의 반복횟수를 조절할 수 있다.

TestFor.java

```
01  public class TestFor {
02      public static void main(String[] args) {
03          int sum = 0;
04          for(int i = 1;   i <= 100 ; i++) {
05              sum += i;
06          }
07          System.out.println("1부터 100까지 정수들의 합계 : "+ sum);
08      }
09  }
```

1부터 100까지 정수들의 합계 : 5050

1.3 while

횟수가 정해지지 않은 반복 처리에 주로 사용되는 반복문으로, for 문과 다르게 반복에 대한 조건식만 제시되며 조건식의 결과가 true 일 동안 수행영역 안의 내용이 반복적으로 실행되는 구조로 동작하는 구문이다. 조건식의 결과가 false 일 때 반복을 종료한다.

```
while (조건식) {
    // 조건식이 참(true)일 경우 실행되는 문장들
}
```

while 문을 for 문처럼 사용할 수도 있는데, 초기식과 증감식을 따로 지정하면 가능하다. 초기식은 while 문 사용 전에 선언하고 증감식은 { } 안에 수행할 내용 마지막에 추가하면 된다.

TestWhile.java

```
01  public class TestWhile {
02      public static void main(String[] args) {
03      int i = 1;     // 초기식
04      sum = 0;
05      while(I <= 100) {     // 조건식
06          sum += i;
07          i++;     // 증감식
08      }
09      System.out.println("1부터 100까지 정수들의 합 : " + sum);
10          }
11  }
```

1.4 do ~ while

do { } 안의 내용을 일단 한번 실행한 다음, 아래에 있는 while 조건식의 결과에 따라 블록 { } 안의 내용에 대한 반복 실행을 결정하는 반복문이다..

```
do {
   // 반복 조건이 참일 때 수행되는 문장들
} while (조건식);      // 마지막에 세미콜론(;)을 반드시 표기해야 한다.
```

TestDoWhile.java

```
01  import java.util.Scanner;
02
03  public class TestDoWhile {
04      public static void main(String[] args) {
05   int value = 0, sum = 0;
06   Scanner in = new Scanner(System.in);
07
08   do {
09          System.out.print("정수 숫자를 입력하세요 (종료는 :99) : ");
10          value = in.nextInt();
11                  sum += value;
12   } while(select != 99); // 루프 마지막에 조건을 비교하여 반복여부를 결정!
13
14              System.out.println("입력된 정수 값들의 합계 : " + sum);
15   System.out.println("프로그램 종료!");
16      }
17  }
```

1.5 중첩 반복문

반복문도 중첩해서 사용할 수 있다. for 문 안에 for 문 또는 while 문 안에 while 문, for 문 안에 while 문, while 문 안에 for 문 등 반복문 종류에 상관없이 다양하게 중첩시킬 수 있다.

TestDupFor.java

```
01  public class TestDupFor {
02      public static void main(String[] args) {
03          for(int i = 2; i <= 9;  i++) {    // 2 ~ 9 까지 열 반복
04              for(int j = 1;  j <= 9;  j++) {  // 1 ~ 9 까지 행 반복
05                  System.out.print (i + "  *  " + j + " = " + (i * j) + "₩t");
06              }
07              System.out.println();
08          }
09      }
10  }
```

```
2 x 1 = 2    2 x 2 = 4    2 x 3 = 6    2 x 4 = 8    …    2 x 9 = 18
3 x 1 = 3    3 x 2 = 6    3 x 3 = 9    3 x 4 = 12   …    3 x 9 = 27
  ….
9 x 1 = 9    9 x 2 = 18   9 x 3 = 27   9 x 4 = 36   …    9 x 9 = 81
```

1.6 break

반복문의 실행 흐름에 변화를 줄 수 있는 문장으로, if 문과 함께 사용되며 break 문을 만나면 해당 반복문이 강제로 종료된다.

TestBreak.java

```
01  public class TestBreak {
02      public static void main(String[] args) {
03    int sum = 0;
04    for(int i = 1;   ;  i++) {   // 조건이 없으면 참으로 판별, 무한 반복
05                        sum += i;
06
07                        if(i == 100)    //  i가 100 일 때 for문 강제 종료
08                            break;
09    }
10
11    System.out.println("1부터 100까지의 정수들의 합 : " + sum);
12       }
13  }
```

break문은 for, while, do-while 모든 반복문에서 사용할 수 있다.

break를 포함하는 반복문 한 개만 종료시킬 수 있다. 중첩 반복문일 경우 범위가 가까운 반복문 한 개만 종료시킨다.

1.7 continue

break 문과 마찬가지로 반복문의 흐름에 변화를 주는 문장이며, 반복할 내용의 중간 생략용으로 이용할 수 있다. 주로 if 문과 함께 사용되며 조건식의 결과가 참일 때 continue 를 실행하며 continue 아래의 반복 수행 구문을 생략하고 for 문일 때는 증감식으로 while 문일 때는 조건식으로 이동한다.

TestContinue.java

```
01  public class TestContinue {
02      public static void main(String[] args) {
03          int sum = 0;
04          for(int i = 1; i <= 100;  i++) {
05              // 홀수들의 합계만 처리할 경우, 짝수는 건너뛰게 처리하려면
06              if(i  %  2 == 0) continue;
07              System.out.print(i + " + ");
08          }
09          System.out.println("₩n 1부터 100 까지의 홀수들의 합 : " + sum);
10      }
11  }
```

1.8 점프 break, continue (Labeled break, continue)

break 와 continue 가 한 개의 반복문만 탈출하는 특성을 보완하기 위해 나온 구문으로 for, while, do-while 문에 사용할 수 있으며, 한 번에 여러 단계의 중첩 반복문을 빠져나가게 할 때 이용할 수 있다. 각각 for 구문 앞, while 구문 앞, do 구문 앞에 이름(Label)을 표시해 두고 사용해야 한다.
이름은 'LABEL 명:' 으로 표시하면 된다.

TestMultiBreak.java

```
01  public class TestMultiBreak {
02      public static void main(String[] args) {
03  label:
04  for(  ;  ;  ) {
05      for(int i = 0;  i < 10;  i++) {
06          System.out.println(i);
07          if( i == 5 ) break label;    // i 값이 5가 되면 반복을 종료
08                  }
09  }
10      }
11  }
```

Chapter 7. 기본자료형 배열

1 배열의 개요

같은 자료형의 변수 여러 개를 하나의 묶음으로 만들어 하나의 이름으로 다루는 것을 배열이라고 한다. 데이터를 저장하기 위해 여러 개의 변수를 선언할 수도 있지만 관리해야 하는 변수가 많아지면 코드가 매우 복잡하고 많아지게 된다. 이러한 경우 하나의 배열이름으로 여러 개의 변수들을 다룰 수 있으면 코드를 간단하게 처리할 수 있게 된다.

자바에서는 배열 공간은 자유 메모리 영역인 Heap 영역에 할당하도록 지정되어 있으며, 배열변수는 할당된 배열공간의 주소를 저장하며, 배열공간의 주소를 이용해 인덱스(순번, index)를 참조하는 방식으로 값들을 처리하도록 정해두었다.

2 배열의 선언과 생성

- **배열의 선언 :** 배열공간의 주소 저장용 참조(Reference) 변수를 만드는 것

```
타입[ ] 변수이름;
타입 변수이름[ ];
```

배열을 선언하면 참조형 변수가 생성된다. 배열을 선언한다고 해서 값을 저장하는 공간이 생기지는 않는다.

```
int[ ] arr;
int arr[ ];
```

- **배열의 생성 :** Heap 영역에 값을 저장하는 변수들을 연속 나열 할당하고, 발생한 배열 공간의 시작주소를 선언된 배열 레퍼런스에 대입한다.

```
배열참조변수 = new 데이터타입 [연속 할당될 변수 갯수] ;
```

배열을 생성할 때 배열의 길이(첨자)를 지정해 주면 해당 데이터타입의 값을 저장할 수 있는 저장공간을 길이 갯수만큼 연속 할당한다. 배열의 선언과 생성은 한 줄로 가능하다.

```
데이터타입 [ ] 변수 = new 데이터타입 [첨자];
```

- 예를 들어, 길이가 3인 문자열(String) 배열을 생성한다면

```
String[ ] strArr = new String[3];
```

3 배열의 초기화

배열 공간 할당과 동시에 초기값을 기록하는 것을 말하며, 배열 초기화시에는 배열의 길이를 따로 지정하지 않고 초기화에 사용되는 값의 개수가 자동 배열의 길이로 처리된다.
자바에서는 배열 공간의 초기화를 따로 지정하지 않아도 자동 초기화 처리가 되도록 각 자료형별 기본값이 준비되어 있다.
기본값은 boolean 은 false, char 은 '₩u0000', 정수형은 0, 실수형은 0.0, 참조형은 null 이다.

```
데이터타입[ ] 배열참조변수 = {값1, 값2, 값3, … };
데이터타입[ ] 배열참조변수 = new 데이터타입[ ] {값1, 값2, 값3, ….};
```

TestArrayInit.java

```
01  public class TestArrayInit{
02  public static void main(String[] args) {
03  String strArr[] = { "Apple", "Banana", "Orange" };
04
05  for(int i= 0; i < 3; i++)
06  System.out.println(strArr[i]);
07  }
08  }
```

4 배열의 인덱스

생성된 배열 공간의 각 저장공간을 요소(element) 라고 부르며, '배열이름 [인덱스]' 형식으로 요소에 접근할 수 있다. 배열의 저장공간의 개수를 배열의 길이(length)라고 하며 , 인덱스는 0 부터 시작해서 배열의 길이 – 1 까지 지정된다.

- **배열의 길이**

배열의 길이는 배열이름.length 를 이용해 알 수 있다.

TestArrayLength.java

```
01  public class TestArrayLength {
02     public static void main(String[] args) {
03        String strArr[ ] = { "Apple", "Banana", "Orange" };
04
05        System.out.println("배열의 길이는 " + strArr.length + "입니다");
06        for(int i= 0; i < strArr.length; i++)
07           System.out.println(strArr[i]);
08     }
09  }
```

5 배열의 복사

배열의 복사는 참조하는 배열공간의 주소를 다른 참조변수에 복사하는 얕은 복사(Shallow Copy) 와 배열공간의 값들을 다른 배열공간에 하나씩 복사하는 깊은 복사(Deep Copy) 가 있다.

TestShallowCopy.java

```
01   public class TestShallowCopy {
02  public static void main(String[] args) {
03  String originArr[ ] = { "Apple", "Banana", "Orange" };
04  String destArr[ ];        // 배열의 주소 복사용 배열레퍼런스 선언
05
06  destArr = originArr;    // 주소 복사
07
08  for(int i = 0;   i< 3;   i++)
09  System.out.println(originArr[i] + " : " + destArr[i]);
10  System.out.println("--------------------");
11
12  originArr[1] = "Pineapple";    // 원본 배열 2번째 인덱스의 값 수정
13
14  for(int i = 0; i < 3; i++)
15  System.out.println(originArr[i] + " : " + destArr[i]);
    }
     }
```

```
Apple : Apple
Banana : Banana
Orange : Orange
--------------------
Apple : Apple
Pineapple : Pineapple
Orange : Orange
```

위와 같이 배열의 주소를 복사한 경우에는 두 개의 배열 레퍼런스가 같은 대상을 공유하게 된다. 별도의 배열 공간을 따로 할당한 다음 원본 배열의 값들을 복사할 경우 for문을 이용해서 직접 인덱스별로 하나씩 복사하는 방법을 사용할 수도 있고, java.lang.System 클래스의 arrayCopy() 메소드를 이용하는 방법이 있다.

- **for문을 이용하여 별도의 배열공간에 하나씩 복사하는 방법**

복사본 배열공간을 따로 생성한 후에 for 문을 이용해 각 인덱스별 데이터를 하나씩 복사한다.

TestDeepCopy.java

```
01  public class TestDeepCopy {
02      public static void main(String[] args) {
03          String originArr[ ] = { "Apple", "Banana", "Orange" };
04          String destArr[ ];
05
06          destArr = new String [originArr.length];
07
08          for(int i= 0;   i< originArr.length;   i++)
09              destArr[i] = originArr[i];
10
11           for(int i= 0;   i< originArr.length;   i++)
12              System.out.println(i + " : " + originArr[i] + ", " + destArr[i]);
13      }
14  }
```

- **arraycopy() 함수를 이용한 복사**

System.arraycopy(src, srcPos, dest, destPos, length) 함수를 이용한 방법

매개변수(src : 원본 배열, src : 원본 배열 시작 인덱스, dest : 복사할 배열, destPos : 복사할 배열 시작 인덱스, length : 복사할 길이)를 지정해서 복사한다. for문을 이용한 복사보다 arraycopy() 함수를 이용한 방법이 더 효율적이다.

java.lang.System.arrayCopy( ) 메소드를 이용한 방법은 다음과 같다.

TestArrayCopy.java

```
01  public class TestDeepCopy {
02      public static void main(String[] args) {
03          String originArr[ ] = { "Apple", "Banana", "Orange" };
04          String destArr[ ] = new String [originArr.length];
05
06          System.arrayCopy(originArr, 0, destArr, 0, originArr.length);
07
08          for(int i = 0;   i < destArr.length;   i++)
09              System.out.println(i + " : " + originArr[i] + ", " + destArr[i]);
10      }
11  }
```

6 다차원 배열

같은 자료형 변수들의 묶음을 1차원 배열이라고 하고, 자료형이 같은 1차원 배열들의 묶음을 2차원 배열이라고 한다. 1차원 배열은 배열기호 [ ] 가 1개이며 2차원 배열은 배열기호가 [ ][ ] 2개이다. 같은 자료형 2차원 배열들의 묶음을 3차원 배열이라고 하며 배열기호는 [ ][ ][ ] 3개로 표시된다. 이러한 2차원 이상의 배열들을 다차원 배열이라고 한다..

- **2차원 배열의 선언**

```
데이터타입 배열참조변수[ ][ ];
데이터타입[ ] 배열참조변수[ ];
데이터타입[ ][ ] 배열참조변수;
```

- **2차원 배열의 생성**

```
배열참조변수 = new 데이터타입[행길이][열길이];     //정변배열
배열참조변수 = new 데이터타입[행길이][ ];          //가변배열, 행만 선언
배열참조변수[행인덱스] = new 데이터타입[열길이];  //행에 대한 열 선언
```

행길이는 묶여질 1차원 배열의 개수를 의미하고, 열길이는 각 1차원 배열의 값 개수를 의미한다. 2차원 배열에서 행길이와 열길이가 같은 경우를 정변배열이라고 하고, 행길이와 행별 열길이가 다른 경우를 가변배열이라고 한다.

- **2차원 배열의 초기화**

```
데이터타입 배열이름[ ][ ] = { {0행0열값, 0행1열값, ... }, {1행0열값, 1행1열값, ...}, ... };
```

TestTwoDimensionalArray.java

```
01  public class TestTwoDimensionalArray {
02      public static void main(String[] args) {
03          int score[ ][ ] = { {1, 2, 3}, {4, 5, 6} };
04
05          for(int i = 0; i < score.length; i++) {
06              for(int j = 0; j < score[i].length; j++) {
07                  System.out.print(i + "행" + j + "열 : " + score[i][j] + "₩t");
08              }
09              System.out.println();
10          }
11      }
12  }
```

2차원 가변배열 초기화

TestTwoVariableArray.java

```
01  public class TestTwoVariableArray {
02      public static void main(String[] args) {
03          int score[ ][ ] = new int[3][ ];
04          score[0] = {1, 2, 3, 4, 5};
05          score[1] = {12, 24, 36};
06          score[2] = {11, 22, 33, 44};
07
08          for(int i = 0; i < score.length; i++) {
09              for(int j = 0; j < score[i].length; j++) {
10                  System.out.print(i + "행" + j + "열 : " + score[i][j] + "₩t");
11              }
12              System.out.println();
13          }
14      }}
```

Part 2. 언어특성 활용하기

Chapter 8. 클래스와 객체

1 객체지향프로그래밍:OOP(ObjectOrientedProgramming)

객체지향 프로그래밍이란 객체를 이용하는 프로그래밍 방식을 말한다. 객체(Object)란 컴퓨터, 고객, 학생, 자동차 등 현실 세계에서 흔히 찾아볼 수 있는 대상을 추상화(Abstraction) 하여 프로그램 상에서 만들어낸 결과물이다. 예를 들어 자동차를 만들어야 한다면, 자동차의 각 부품들을 별개로 제작한 다음 필요한 부품을 조립하는 방식으로 자동차를 완성해 나가는 것이 객체지향 프로그래밍 방식이라고 보면 된다.
객체 지향 프로그래밍의 기술적 특징은 추상화(Abstraction), 캡슐화(EnCapsulation), 상속(Inheritance), 다형성(Polymorphism) 이 있다. 이 기술들을 이용하여 프로그램을 구현하는 것이 객체지향 프로그래밍이다.

- **추상화(Abstraction) :** 프로그램이 필요로 하는 실제 데이터들을 모델링하는 기술
- **캡슐화(EnCapsulation)** : 객체 지향의 가장 중요한 데이터 보호 기술
- **상속(Inheritance)** : 클래스의 재사용과 소스 코드의 중복 제거를 목적으로 하는 기술
- **다형성(Polymorphism)** : 상속을 이용하여 여러 클래스 타입을 하나의 클래스 타입으로 다루는 기술

1.1 추상화(Abstraction)와 캡슐화(Encapsulation)

프로그램에서 필요로 하는 데이터와 데이터를 처리하기 위한 동작들을 정리하는 과정을 추상화라고 하며, 추상화 과정에서는 소스 코드의 구현이 아닌 데이터들의 객체화와 객체들 간의 관계에 대한 모델링 과정이 중요하다.
클래스(class)는 추상화한 내용을 정리한 설계도와 같은 것이고, 객체는 클래스를 실제 사용한 프로그램 상에서의 결과물이라고 보면 된다.

객체지향 관점에서의 캡슐화는 여러 자료형 변수들을 class 영역 안에서 젭근제한자 private 을 사용하여 멤버변수(Field)로 선언하여 외부에서 클래스 내부의 데이터에 접근하지 못하도록 막는 것이 핵심이다. 이렇게 클래스 내부의 데이터를 외부에서 접근하지 못하도록 설정함으로써 객체가 가진 데이터를 보호할 수 있게 되며, 이를 정보 은닉(Information Hiding)이라고 한다.

2 클래스(class)와 객체(Object == Instance)

2.1 클래스 (class)

객체를 만들기 위해 필요한 객체의 설계도라고 볼 수 있다. 자료형이 다른 변수들을 배열처럼 메모리에 연속 나열 할당하기 위한 순서를 지정하는 역할을 하며, 클래스 블록 { } 이 캡슐 즉 보호막의 역할을 하여 객체 외부에서 멤버변수(Field)로의 접근을 제한하며, 멤버변수들을 하나의 타입으로 묶어 주는 것이 클래스이다.

2.2 객체(object)와 인스턴스(instance)

추상화가 이루어진 클래스로부터 데이터가 존재하는 실체로 만드는 과정을 인스턴스화(instantiation)라고 하며, 클래스로부터 만들어진 객체를 해당 클래스의 인스턴스(instance)라고 한다. 자바에서는 객체, 즉 인스턴스는 무조건 메모리 힙(Heap) 영역에 할당하도록 정하였으며 할당된 객체의 주소를 기록할 참조(Reference)변수를 사용하여 객체를 참조하도록 정해 두었다.

3 접근 제한자 (Access Modifier)

클래스에 속한 멤버변수(Field)와 멤버함수(Method) 들을 클래스 외부(다른 클래스)에서의 접근에 대한 제한을 걸기 위해 사용되는 단어를 말한다.

3.1 접근 제한자의 종류

- (+) public : 클래스 안과 밖, 패키지 안과 밖에 있는 모든 클래스들이 접근 가능하다.
- (#) protected : 비 상속시에는 default 와 동일하며, 같은 패키지에 속한 클래스 간에만 멤버의 접근이 가능하며 다른 패키지에 있는 클래스일 때는 상속관계의 후손클래스 내에서만 부모클래스의 멤버에 접근이 가능하다.
- (~) default (package private) : 같은 패키지에 속한 클래스 간에만 멤버들에 대한 접근이 가능하다.
- (-) private : 멤버변수의 캡슐화를 위해 사용되는 제한자로 클래스 안에서만 접근이 가능하며 클래스 밖에서는 접근이 불가능하다.

접근 제한자의 공개 범위는 public > protected > default > private 순으로 좁아지며, protected 접근 제한자는 상속관계가 아니면 잘 사용하지 않는다.

- **private 필드 접근**

access.sample.TestPrivate.java

```
01  package access.sample;
02
03  public class PrivateSample {
04      private int num;    // 선언된 클래스 안에서만 접근 가능
05      public int getNum() {  // 값 호출을 위한 Getter
06          return num;
07      }
08      public void setNum(int num) {  // 값 대입을 위한 Setter
09          this.num = num;
10      }
11  }
```

access.test.TestPrivate.java

```
01  package access.test;
02
03  public class TestPrivate {
04      public static void main(String[] args) {
05          PrivateSample psamp = new PrivateSample();
06
07          psamp.num = 10;    // private 이므로 접근 제한 에러 발생
08
09          psamp.setNum( 10 ); // public 이므로 접근 가능
10          System.out.println( psamp.getNum( ) );    // public 이므로 접근 가능
11      }
12  }
```

- **public 필드 접근**

access.sample.PublicSample.java

```
01  package access.sample;
02
03  public class PublicSample {
04      public int num;
05  }
```

access.test.TestPublic.java

```
01  package access.test;
02
03  public class TestPublic {
```

```
04      public static void main(String[] args) {
05          PublicSample pbt = new PublicSample();
06
07          pbt.num = 10; //  public 이므로 접근 가능
08          System.out.println( pbt.num ); // 접근 가능
09      }
10  }
```

- **protected 필드 접근**

access.sample.Parent.java

```
01  package access.sample;
02
03  public class Parent {
04      protected int num;
05  }
```

access.sample.Child.java

```
01  package access.sample;
02
03  public class Child extends Parent {
04      public void insert() {
05          num = 10;          //protected 멤버는 상속된 후손 내에서만 사용 가능함
06      }
07      public void out( ) {
08          System.out.println(num);
09      }
10  }
```

access.test.TestProtected.java

```
01  package inherit.test;
02
03  import inherit.sample.Child;
04  import inherit.sample.Parent;
05
06  public class TestProtected {
07      public static void main(String[] args) {
08          Parent parent = new Parent();
09          Child child = new Child();
10
11          parent.num = 10; // protected 멤버는 다른 패키지의 클래스 밖에서는 접근 불가
12          child.insert ( );
```

```
13          child.out ( );
14       }
15    }
```

- **default 필드 접근**

access.sample.DefaultSample1.java

```
01    package access.sample;
02
03    public class DefaultSample1 {
04       int num1;    //default 는 단어로 명시되지 않음. 접근제한자가 생략되면 default 임
05    }
```

access.test.DefaultSample2.java

```
01    package access.test;
02
03    public class DefaultSample2 {
04       int num2;
05    }
```

access.test.TestDefault.java

```
01    package access.test;
02
03    import access.sample.DefaultSample1;   // 다른 패키지에 속한 클래스는 import 해야 사용 가능
04
05    public class TestDefault {
06       public static void main(String[] args) {
07          DefalutSample1 dt1 = new DefalutSample1( );
08          DefaultSample2 dt2 = new DefaultSample2( );
09
10          dt1.num1 = 100; // 다른 패키지의 클래스에서는 접근 불가능
11          dt2.num2 = 100; // 같은 패키지의 클래스에서는 접근 가능
12       }
13    }
```

4 클래스 구조

클래스는 클래스 해더(Header) 시그니처(Signature)와 클래스 바디(Body)로 구성된다.

클래스 해더 시그니처는 클래스의 이름과 종류를 지정하는 부분이고, 클래스 바디 안에는 클래스의 멤버들이 정의되는 부분이다.

4.1 클래스 헤더

클래스 작성시 클래스의 이름과 종류를 정의하는 클래스 시그니처는 다음과 같다.

[접근제한자] [클래스식별자] class 클래스이름

- **접근제한자**

클래스의 접근에 제한을 두는 예약어로 public 과 default (package private) 두 가지가 사용된다.

public class 클래스명 : 패키지 안/밖 모두 접근(사용) 가능한 클래스를 의미한다.

class 클래스명 : 선언된 패키지 안에서만 접근(사용) 가능한 클래스를 의미한다.

클래스 식별자 (Identifier)

클래스의 종류를 지정하는 예약어로 final, abstract 두 가지가 사용될 수 있다.

public final class 클래스명 : 종단 클래스, 더 이상 상속시킬 수 없는 클래스, 서브(후손) 클래스를 만들 수 없다

public abstract class 클래스명 : 추상(미완성된) 클래스, 상속을 이용해 후손 클래스가 부모의 미완성된 기능을 완성시킨다.

- **x, y좌표를 이용해 점을 표현하는 Point 클래스**

oop.sample.Point.java

```
01  package oop.sample;
02
03  public class Point {  // 클래스 헤더
04    private int posX; // 멤버필드
05    private int posY; // 멤버필드
06
07    // 메소드
08    public int getPosX() {          return posX;       }
09    public void setPosX(int posX) {          this.posX = posX;       }
10    public int getPosY() {          return posY;       }
11    public void setPosY(int posY) {          this.posY = posY;       }
12
13    // 생성자
14    public Point ( ) { }
15    public Point (int posX, int posY) {
```

```
16        this.posX = posX;
17        this.posY = posY;
18     }
19  }
```

oop.sample.TestPoint.java

```
01  package oop.test;
02
03  import oop.sample.Point;
04
05  public class TestPoint {
06      public static void main (String [ ] args) {
07          Point p1 = new Point ( );
08          Point p2 = new Point (10, 20);
09
10          System.out.println ("p1 : " + p1.getPosX( ) + ", " + p1.getPosY( ));
11          System.out.println ("p2 : " + p2.getPosX( ) + ", " + p2.getPosY( ));
12
13          p1.setPosX (12);
14          p1.setPosY (23);
15          System.out.println ("p1 : " + p1.getPosX( ) + ", " + p1.getPosY( ));
16      }
17  }
```

4.2 멤버 변수 (Field)

클래스에 소속된 멤버 변수와 멤버 상수를 필드(Field)라고 하며, 클래스 내부에 선언한다

접근제한자 식별자 자료형 변수명 [= 초기값];

- **변수(Variable) 의 종류**

① **인스턴스(Instance) 변수**

접근제한자 자료형 변수명;

클래스 { } 안에 선언되며, 클래스에 대한 객체(Instance == Object)를 동적 메모리에 할당할 때 객체 공간 안에 생성되는 필드들을 말한다.

```
public class Var1 {
   private int num1;     // 인스턴스(Instance) 멤버 변수
}
```

② **클래스 변수**

클래스 { } 안에 선언되며 멤버 변수 선언시 자료형 앞에 static 키워드를 붙여 선언한 필드를 말하며, 해당 클래스형 객체들이 모두 공유하는 필드가 된다.

접근제한자 static 자료형 필드명;

```
public class Var2 {
  private static int num2; //  static(정적) 멤버변수 (클래스 변수)
  // Var2 클래스에 의해 생성된 모든 객체가 num2 필드를 공유하게 된다.
}
```

③ **지역 변수 (Local Variable)**

메소드 { } 내에 선언된 변수로 메소드 내에서만 사용이 가능하며, 메소드 실행시 메모리에 생성되었다가 메소드가 종료시 메소드와 같이 메모리에서 소멸되어 사용할 수 없게 된다.

```
public class Var3 {
  public void method3(){
    int num3 = 10;     // method3 메소드 안의 지역 변수임
  }
}
```

4.3 멤버함수 (Method)

클래스 { } 내에 작성되는 함수로 멤버변수들에 대한 기능 및 동작을 정의한다.

- **메소드의 종류**

① **인스턴스 메소드 (non-static method)**

클래스에 소속된 일반 메소드로, 클래스에 대한 객체의 주소(레퍼런스)를 전달받아 해당 주소위치에 있는 객체의 인스턴스 변수 값들을 연산 처리한다.

정의 : 접근제한자 반환자료형 메소드명 ([자료형 매개변수명, ...])

호출 : [자료형 변수명 =] 레퍼런스.메소드명 ([전달값, ...]);

② **클래스 메소드 (static method)**

클레스 멤버함수 작성시 메소드 반환자료형 앞에 static 키워드를 사용한 메소드이다. 인스턴스 메소드와 다르게 메소드 호출시 객체의 레퍼런스를 사용하지 않고 클래스명을 사용하여 호출한다.

정의 : 접근제한자 static 반환자료형 메소드명 ([자료형 매개변수명, ...])

호출 : [자료형 변수명 =] 클래스명.메소드명 ([전달값, ...]);

```
[접근제한자] [식별자] [리턴타입] [메소드명] (매개변수1, 매개변수2, ... ) {     // 메소드 해드
        // 메소드 Body : 동작, 기능
        // 처리내용에 대한 코드 구현부
        return [값];
}
```

4.3.1 메소드 시그니처(Signature) 선언

메소드 선언부에는 메소드의 접근제한자, 메소드의 리턴타입, 메소드 이름, 매개변수를 선언한다.

4.3.2 리턴 타입

프로그램 구동시 모든 메소드들은 메소드 내의 소스 코드에 대한 실행이 끝나면 해당 메소드가호출된 위치로 되돌아 가도록 되어 있다. 메소드에서 호출 위치로 되돌아 갈 때 메소드의 수행결과를 가지고 되돌아 갈 수 있는데 이 결과값을 반환값이라고 한다.

메소드가 결과값을 반환하는 경우에는 메소드 해드의 반환 자료형을 이 리턴값의 자료형과 반드시 일치시켜야 하며, 반환값이 없는 경우에는 리턴타입은 void 라고 명시한다.

4.3.3 메소드 이름

해당 메소드를 호출할 때 사용하는 소스 코드 묶음의 이름이며, 자바의 Naming Rule 규칙에 맞게 작성한다.

4.3.4 메소드의 매개변수(parameter) 선언

메소드는 매개변수가 없는 경우와 매개변수가 있는 경우로 구분할 수 있으며, 매개변수가 있는메소드일 때 일반 변수 선언과 동일하게 (자료형 변수명)으로 선언한다. 매개변수가 있는 메소드일 경우 메소드 호출시 매개변수에게 넘길 전달값(argument)을 메소드 ( ) 안에 기입하여야 한다.

4.3.5 return 문

메소드 구현부 맨 마지막에는 return 문이 무조건 존재한다. 메소드가 호출 위치로 리턴될 때 반환값없이 리턴될 때에는 return; 으로 표현되며 생략될 수도 있다. 반환값이 있는 리턴일 경우에는 return 값; 으로 표시하며 메소드 해드의 반환자료형과 반환값의 자료형은 반드시 같아야 한다.

4.3.6 메소드 오버로딩 (Overloading)

하나의 클래스 영역 내에서 이름이 같은 메소드를 여러 개 정의할 수 있다.. 이러한 경우에 메소드가 오버로딩되었다고 표현하며, 동일한 기능을 수행하는 메소드가 전달값을 각각 다르게 받아서 구동되는 경우 매번 메소드의 이름을 다르게 작성하는 것보다 훨씬 더 프로그램 가독성을 좋게 할 수 있다.

메소드 오버로딩시의 규칙은 메소드 이름은 반드시 같아야 하며, 매개변수의 자료형과 개수는 다르게 구성되어야 한다.

oop.method.sample.OverloadingSample.java

```
01  package oop.method.sample;
02
03  public class OverloadingSample {
04    public void out ( ) {
05      System.out.println ("out ( ) 메소드 실행!");
06    }
07    public void out (int num) {  // 메소드 이름은 같고 매개변수가 다르다. 오버로딩 성립!
08      System.out.println ("out (" + num + ") 메소드 실행!");
09    }
10  }
```

oop.method.test.TestOverloading.java

```
01  package oop.method.test;
02
03  public class TestOverloading {
04    public static void main (String [ ]  args) {
05      OverloadingSample osamp = new OverloadingSample( );
06      osamp.out ( );
07      osamp.out (123);
08    }
09  }
```

4.4 클래스의 객체(Object == Instance) 생성

자바에서는 선언된 클래스를 프로그램에서 사용하고자 할 때, 해당 클래스에 대한 객체(Object)를 동적 메모리에 new 키워드를 사용해 할당하도록 정하였다. 이렇게 할당된 객체의 위치(주소) 를 해당 클래스 타입의 레퍼런스 변수에 기록하고, 레퍼런스가 가진 주소를 통해 객체에 접근해서 객체 안의 인스턴스변수 값들을 처리하는 방식의 객체지향 프로그래밍 방법을 사용한다.

- **Point 클래스를 이용한 객체 생성과 필드 값 출력 처리 예**

oop.sample.vo.Person.java

```
01  package oop.sample.vo;
02
03  public class Person {
04    private String name;
05    private int age;
06    private char gender;
07
```

```
08      public Person ( ) { }
09      public Person (String name, int age, char gender) {
10          this.name = name;
11          this.age = age;
12          this.gender = gender;
13      }
14
15      public void display( ) {
16          System.out.println(name + ", " + age + "세, " + gender + "자");
17      }
18  }
```

oop.test.TestPerson.java;

```
package oop.test;

import oop.sample.vo.Person;

public class TestPerson {
    public static void main (String [ ] args) {
        Person p1 = new Person( );
        Person p2 = new Person("홍길동", 25, '남');

        p1.display ( );
        p2.display ( );
    }}
```

4.4.1 생성자 (Constructor)

클래스에 대한 객체 생성시 인스턴스변수의 초기화를 담당하는 메소드이다.

클래스명 레퍼런스변수명 = new 생성자 ([초기값, ...]);

생성자는 이름이 반드시 클래스명과 동일해야 하고 리턴타입이 없다.
생성자는 오버로딩 가능하므로 매개변수 타입과 개수를 다르게 하여 여러 개의 생성자를 만들 수 있다.

```
[접근제한자] 생성자이름 (자료형 매개변수,  ... ) {
    this.필드명 = 매개변수;  //매개변수가 전달받은 초기값으로 인스턴스변수를 초기화함
    ..............
}
```

4.4.2 기본(Default) 생성자

매개변수가 없는 생성자로 객체 생성시 JVM 이 준비한 기본값으로 초기화된다.
(기본값 : Boolean – false, 정수 – 0, 실수 – 0.0, char – '₩u0000', 참조형 – null)
클래스에는 반드시 하나 이상의 생성자가 정의되어 있어야 한다. 클래스에 생성자가 작성되어 있지 않을 경우에는 JVM에 의해 컴파일시에 자동으로 추가되는 생성자이다.
매개변수가 있는 생성자가 한 개 작성된 경우에는 기본 생성자는 자동 추가되지 않는다.

- **Point 클래스의 기본(Default) 생성자**

```
public class Point {
        // 기본 생성자 (Default Constructor)
        public Point() { }  //JVM 기본값으로 초기화됨
}
```

Point 클래스의 posX필드와 posY필드에 대해서 각각 0과 0으로 초기화 된다.

4.4.3 매개변수가 있는 생성자

생성자는 오버로딩(Overloading)이 가능하므로 생성자의 매개변수를 다르게 구성하여 여러 개의 생성자를 만들 수 있다. 매개변수가 있는 생성자와 디폴트 생성자 둘 다 필요하다면 둘 다 정의해야 한다

- **Point 클래스의 매개변수 있는 생성자**

```
public class Point {
        private int posX, posY;

        // 기본 생성자 (Default Constructor)
        public Point() { }  //JVM 기본값으로 초기화됨

        // 매개변수 있는 생성자
        public Point(int posX, int posY) {
                this.posX = posX;
                this.posY = posY;
        }
}
```

4.5 this 레퍼런스변수와 this( ) 생성자

4.5.1 this 레퍼런스변수

자기 참조 변수라고 하며, 클래스 내에 작성 정의된 필드나 메소드에 접근 가능한 레퍼런스변수로 프로그램 구동시 할당된 객체의 주소를 전달받아 해당 주소 위치에 있는 객체의 인스턴스변수에 접근할 수 있는 레퍼런스변수이다. 해당 클래스의 인스턴스 메소드 내부에 존재하며 해당 메소드 호출시 자동으로 참조할 객체의 주소를 전달받도록 되어 있다.

4.5.2 this ( ) 생성자

생성자 내에서 해당 클래스의 다른 생성자를 호출할 때 사용할 수 있다
사용할 경우에는 반드시 생성자 { } 안의 첫 줄에 기입해야 한다.
생성자의 처리 코드가 중복되었을 때 코드 중복을 제거할 목적으로 사용할 수 있다

oop.sample.Point.java

```
01  package oop.sample;
02
03  public class Point {
04     private int posX; // 멤버필드
05     private int posY; // 멤버필드
06
07     // 메소드
08     public void out() {
09        System.out.println("[" + posX + ", " + posY + "]");
10     }
11
12     // getter, setter 메소드
13     public int getPosX() {          return posX; }
14     public void setPosX(int posX) {          this.posX = posX; }
15
16     public int getPosY() {          return posY; }
17     public void setPosY(int posY) {          this.posY = posY; }
18
19     // 디폴트 생성자
20     public Point() {
21        this (10, 20);
22     }
23
24     // 매개변수가 있는 생성자
25     public Point(int posX, int posY) {
26        this.posX = posX;
27        this.posY = posY;
       }
    }
```

oop.test.TestThis.java

```
01  package oop.test;
02
03  import oop.sample.Point;
04
05  public class TestThis {
06     public static void main(String[] args) {
07        Point p = new Point();
08
09        System.out.println(p.getPosX());
10        System.out.println(p.getPosY());
11     }
12  }
```

```
[10, 20]
[100, 200]
```

Chapter 9. 객체배열

1 객체배열 이란

많은 수의 객체를 배열로 관리하는 것으로 객체를 배열로 선언하여 생성한 후 각 인덱스에 new연산자를 통해 서객체를 생성하여 대입하고 배열의 인덱스를 통하여 객체에 접근하여 수정 / 활용하는 것을 말한다.

2 객체배열의 선언

객체배열의 선언은 기본자료형 배열의 선언과 비슷하다. 아래 예시와 같이 기본자료형이 들어가는 부분에 사용자 정의 자료형 즉 객체명을 선언하면 된다.

```
클래스명 배열명[ ] = new 클래스명 [객체 수] ; //객체 배열선언
```

3 객체배열의 대입

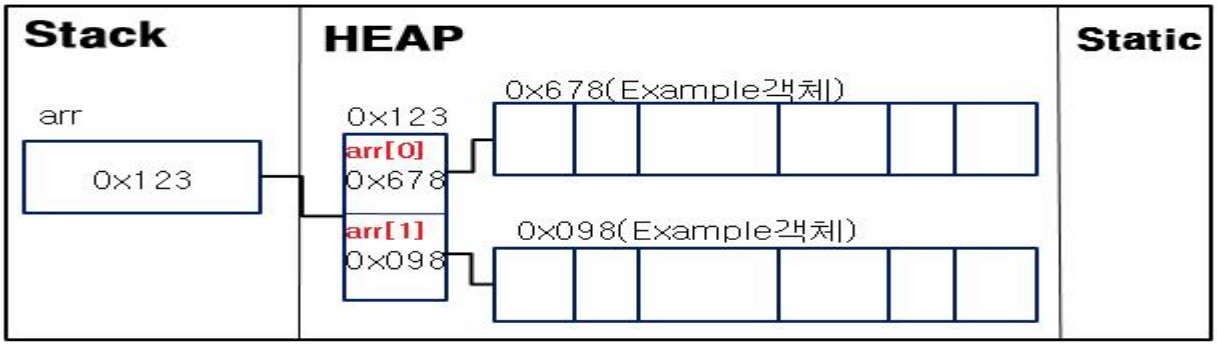
객체배열 선언 후 객체를 대입할때에는 new라는 연산자를 통해서 객체를 생성한 후 배열의 인덱스에 대입하는 방법이 있고, { } 안에서 쉼표(,)를 구분자로 저장할 수 있다.

```
//인덱스를 이용하여 대입
객체배열명[숫자] = new 클래스명();
//선언과 동시에 대입
클래스명 배열이름[] = {new 클래스명(), new 클래스명(),.....};
```

4 객체배열의 구조

객체배열을 선언하면 레퍼런스형(사용자 자료형)을 보관할 수 있는 선언한 배열명의 변수가 생성이 되고 이 변수는 new연산자를 통해 생성된 배열을 값으로 가지고 있음.

new 연산자로 생성된 배열은 객체를 보관할수 있는 레퍼런스형(사용자 자료형) 변수를 만들고 배열이기 때문에 각 변수에 인덱스 번호를 할당한다. 이 인덱스가 할당된 변수안에 생성되는 객체가 저장되는 구조이다.



객체배열에는 객체를 보관하기 때문에 배열안에 값을 넣을때는 반드시 new연산자를 통해서 생성한 뒤에 대입해야한다.

5 객체배열의 호출

객체 배열도 하나의 배열이기 때문에 자료에 접근할때는 인덱스를 활용하여 접근을 한다. 변수명[인덱스]로 객체에 접근하고 다시 해당 객체의 멤버변수나, 메소드에 접근하려면 접근연산자 " . "을 이용하여 접근한다. 간단히 얘기하면 변수명[인덱스]를 하나의 객체변수로 생각하면 된다. 그렇게 생각한 후 객체변수가 자신의 멤버변수와 메소드에 접근한 방법을 생각하면 쉽게 자료에 접근할 수 있을것이다.

oop.sample.Person.java

```
01  package oop.sample;
02
03  public class Person {
04    private String name; // 멤버필드
05    private int age; // 멤버필드
06    private char gender; // 멤버필드
07
08    // 기본 생성자
09    public Person() {}
10
11   // 매개변수 있는 생성자
12    public Person (String name, int age, char gender) {
13            this.name=name;
14            this.age=age;
15            this.gender=gender;
16     }
17
18    // getter, setter 메소드
19    public char getName() {      return name; }
```

```
20      public void setName(String name) {    this.name= name; }
21
22      public int getAge() {           return age; }
23      public void setAge(int age) {           this.age = age; }
24
25      public char getGender() {    return gender; }
26      public void setGender(char gender) {  this.gender= gender; }
```

oop.test.TestArray.java

```
01  package oop.test;
02
03  import oop.sample. Person;
04
05  public class TestArray {
06      public static void main(String[] args) {
07          Person[] list = new Person[3]; // 3명을 보관할 수 있는 객체배열 선언
08          list[0]=new Person("김개똥", 19, '남');
09          list[1]=new Person("김말똥", 20, '남');
10          list[2]=new Person("김소똥", 22, '여');
11          //각 객체의 멤버에 접근 이름출력
12          for(int i=0; i<list.length;i++){
13              System.out.println(list[i].getName());
14          }
15      }
    }
```

```
김개똥
김말똥
김소똥
```

Chapter 10. 상속

1 상속 (inheritance)

기존의 class(부모 클래스/ 슈퍼 클래스)를 재사용하여 새로운 class(자식클래스/ 서브클래스)를 작성하는 것을 말한다. 기존 부모 클래스의 필드와 메소드를 내려 받고, 여러 후손 클래스들의 중복되는 멤버들은 부모 클래스에서 작성 관리하므로써 멤버의 중복도 줄이면서, 적은 양의 코드로도 다양한 기능을 처리할 수 있는 새로운 클래스를 만들 수 있다.

자바에서는 멤버의 모호성을 없애기 위해, 클래스 간의 다중상속은 허용하지 않는다.

2 상속 지정

자바에서 클래스 간의 상속을 지정할 때는 새로 만들어지는 클래스이름 뒤에 extends 키워드를 사용하고 extends와 함께 상속받을 부모 클래스이름을 기입한다.

```
접근제한자 class 클래스이름 extends 부모클래스명 {
    // 멤버 정의
  }
```

```
public class A {
    private int num;
    public void method1() { ... }
  }
public class B extends A {
    private String str;
    public void method2() { ... }
  }
```

extends 키워드를 이용해 A 클래스와 B 클래스는 상속관계를 맺었다. 여기서 B 는 자식 클래스가 되고 A 는 부모 클래스가 된다. B 클래스는 A 클래스의 필드 num과 메소드 method1() 를 상속받아 직접 구현하지 않았어도 자신이 가지고 있는 것처럼 사용이 가능하다.

단, 부모의 private 멤버는 상속은 받지만 접근은 불가능하다.

2.1 부모(super) 클래스

상속하는 클래스를 슈퍼(super) 클래스라고 한다

2.2 자식(sub) 클래스

상속 받는 클래스를 후손/하위/자식/파생 클래스라고 한다

3 메소드 오버라이딩 (Overriding)

메소드 재정의라고 하고, 부모 클래스로부터 상속받은 메소드를 자식 클래스에서 다시 정의하는 것을 말하며 후손 클래스에서 부모 메소드를 재정의할 때 메소드 상단에 '@Override' 어노테이션을 사용하여 재정의한다는 것을 표시해 준다.

test.inherit.TestOverriding.java

```
01 package test.inherit;
02
03 public class TestOverriding {
04    public static void main(String[] args) {
05       Parent p = new Parent();
06       p.display();
07       // p.out()         //부모객체는 자식객체의 멤버에 접근할 수 없다
08
09       Child c = new Child();
10       c.out();
11       c.display();
12    }
13 }
```

부모 클래스 메소드
상속받아 재정의한 메소드
자식 클래스 메소드

inherit.sample.Parent.java

```
01 package inherit.sample;
02
03 public class Parent {
04    private int num;
05
06    void display() {
07       System.out.println("부모 클래스 메소드");
08    }
09 }
```

inherit.sample.Child.java

```
01 package inherit.sample;
02
```

```
03  public class Child extends Parent {
04      public void out() {
05          System.out.println("자식 클래스 메소드");
06      }
07      @Override
08      public void display() {
09          System.out.println("상속받아 재정의한 메소드");
10      }
11  }
```

3.1 정적바인딩 vs 동적바인딩

메소드에 대한 호출과 실제 구현된 메소드의 코드와 연결되는 것을 바인딩(binding)이라고 한다. 바인딩에는 크게 정적 바인딩과 동적 바인딩이 있다.

정적 바인딩은 객체의 타입이 컴파일러에 의해 컴파일되는 시점에 결정되는 것을 말하며 private, final, static 메소드의 경우 모두 정적 바인딩이 된다.

동적 바인딩은 객체의 타입이 실행 중에 결정되는 것을 말하며, 상속관계의 오버라이딩된 메소드가 동적바인딩을 한다. 오버라이딩된 메소드를 호출하는 코드가 존재할 경우 컴파일 시에는 단지 문법적으로 타입 에러만을 체크하고, 실제 프로그램 동작 중에 레퍼런스가 참조하는 객체의 타입이 결정되어 호출된 오버라이딩 메소드가 부모클래스의 메소드인지 자식클래스의 메소드인지 판단하여 동작하게 된다.

- 적 바인딩 예 (Parent.java와 Child.java 소스는 이전 예제와 동일함)

inherit.bind.TestBinding.java

```
01  package inherit.bind;
02
03  public class TestBinding {
04      public static void main(String[] args) {
05          Parent p = new Child();
06
07          p.display( );  // 컴파일시에는 정적 바인딩, 실행시에는 동적 바인딩됨
08      }
09  }
```

```
상속받아 재정의한 메소드
```

상속관계에 있는 클래스는 부모 클래스타입 레퍼런스변수에 자식클래스의 인스턴스의 주소를 기록할 수 있다. 이 경우 p 레퍼런스를 통해 display() 메소드를 호출할 때 정적 바인딩이 이루어진다면 p 레퍼런스로는 Parent 클래스의 display() 메소드만 호출할 수 있게 된다. 하지만 display 메소드는 오버라이딩 되어 있기 때문에 동적 바인딩이 되므로, p 레퍼런스가 참조하는 객체의 타입이 Child 클래스이면, 후손의 오버라이딩된 display() 메소드가 실행되게 되는 것이다.

3.2 super 레퍼런스와 super() 생성자

자식클래스에서 부모클래스의 생성자나 멤버에 접근하고 싶을 때 super 키워드를 사용한다
super()는 부모 클래스의 생성자를 호출하는데 사용할 수 있으며 자식클래스에서 생성자를 정의할 때 사용하면 코드의 재사용성을 높일 수 있다

- super : 부모 클래스의 멤버를 지칭할 때 사용
- super() : 부모 클래스의 생성자를 호출할 때 사용. 단 반드시 후손 생성자 { } 안 첫 줄에 사용해야 한다.

inherit.test.TestSuper.java

```
01 package inherit.test;
02
03 public class TestSuper {
04    public static void main(String[] args) {
05       Child2 c = new Child2();
06       c.out();
      }
   }
```

```
부모 생성자
자식 생성자
부모 메소드
자식 메소드
```

inherit.sample.Parent2.java

```
01 package inherit.sample;
02
03 public class Parent2 {
04    public Parent2() {
05       System.out.println("부모 기본 생성자");
06    }
07
08    public void out() {
09       System.out.println("부모 out() 메소드");
      }
   }
```

inherit.sample.Child2.java

```
01  package inherit.sample;
02
03  public class Child2 extends Parent2 {
04      public Child2() {
05          super();            // 부모 기본 생성자 호출
06          System.out.println("자식 기본 생성자");
07      }
08
09      @Override
10      public void out() {
11          super.out();        // 부모 메소드 out() 호출
12          System.out.println("자식 out() 메소드");
13      }
14  }
```

4 final 예약어

더 이상 확장이 불가능함을 알리는 예약어로, 지역변수, 필드, 메소드, 클래스에 붙일 수 있으며 각각 의미하는 바가 다르다.

4.1 final 변수

변수에 기록된 값을 변경할 수 없도록 한다. 즉, 변수를 상수화시킨다
상수화시키면 초기값도 변경할 수 없으므로, 반드시 선언과 함께 초기화도 하여야 한다

```
final 데이터타입 변수명 = 초기화값;
```

test.keyword.TestFinalVariable.java

```
01  package test.keyword;
02
03  public class TestFinalVariable{
04      public static void main(String[] args) {
05          final int num = 10;
06          num = 100;      // 에러. 변경 불가
07      }
08  }
```

4.2 final 메소드

상속받은 후손 클래스에서 부모의 메소드를 오버라이딩할 수 없게 한다.

```
접근제한자 final 리턴타입 메소드이름 (자료형 매개변수) { ... }
```

```
public class Parent {
    public final void out() {
        System.out.println("final 메소드");
    }
}

public class Child2 extends Parent2 {
    @Override
    protected void out() {      // 에러 발생
    }
  }
```

4.3 final 클래스

종단 클래스라고 하며, 더 이상 상속에 사용할 수 없는 클래스라고 지정할 때 사용한다. 즉 부모 클래스가 될 수 없는 후손 클래스를 파생시킬 수 없는 클래스라는 의미가 된다.

```
접근제한자 final class 클래스이름 { ... }
```

```
public final class Parent {
  private int num;
}

public class Child extends Parent {        // 에러발생
    // ...
  }
```

inherit.sample.Phone.java

```
01  package inherit.sample;
02
03  public class Phone {
04      private String model;
```

```
05      private String number;
06
07      public Phone () {}
08      public Phone(String model, String number) {
09          this.model = model;
10          this.number = number;
11      }
12
13      public String getModel() {
14          return model;
15      }
16      public void setModel(String model) {
17          this.model = model;
18      }
19      public String getNumber() {
20          return number;
21      }
22      public void setNumber(String number) {
23          this.number = number;
24      }
25  }
```

inherit.sample.IPhone.java

```
01  package inherit.sample;
02
03  public class IPhone extends Phone {
04      private int memorySize;
05
06      public IPhone () {}
07      public IPhone(String model, String number, int memorySize) {
08          super(model, number);
09          this.memorySize = memorySize;
10      }
11
12      public void setMemorySize (int memorySize) {
13          this.memorySize = memorySize;
14      }
15      public int getMemorySize () {      return memorySize;    }
16  }
```

inherit.sample.GPhone.java

```
01  package inherit.sample;
02
03  public class GPhone extends Phone {
04      private String os = "Android";
05
06      public GPhone () {}
07      public GPhone(String model, String number) {
08          super(model, number);
09      }
10
11      public String getOs() {
12          return os;
13      }
14      public void setOs(String os) {
15          this.os = os;
16      }
17  }
```

test.inherit.PhoneMain.java

```
01  package test.inherit;
02
03  import inherit.sample.*;
04
05  public class PhoneMain {
06      public static void main(String[] args) {
07          IPhone iPhone = new IPhone("7", "010-xxxx-xxxx", 32);
08          GPhone gPhone = new GPhone("Vega", "010-yyyy-yyyy");
09
10          System.out.println(iPhone.getModel() + ", " + iPhone.getNumber()
11                  + ", " + iPhone.getMemorySize());
12          System.out.println(gPhone.getModel() + ", " + gPhone.getNumber()
13                  + ", " + gPhone.getOs());
14          }
15  }
```

```
7, 010-xxxx-xxxx, 32
Vega, 010-yyyy-yyyy, Android
```

Chapter 11. 다형성

1 다형성 (polymorphism)

객체지향개념에서 다형성이란 '**여러가지 타입을 한가지 타입으로 처리할 수 있는 기술**'을 의미하며, 자바에서는 한 타입의 참조변수로 여러타입의 객체를 참조할 수 있도록 하였다. 즉 **부모클래스 타입의 참조변수로 자식클래스의 인스턴스를 참조할 수 있다.**

생성한 인스턴스를 다루기 위해, 인스턴스의 타입과 일치하는 타입의 참조변수만 사용하기도 하지만, 상속관계의 클래스에서는 자식클래스타입의 인스턴스를 생성할 때, 부모클래스 타입의 참조변수를 사용할 수 있다.

Parent.java

```
01 package poly.sample;
02
03 public class Parent {
04    protected int num;
05
06    public void display() {
07       System.out.println("부모 클래스 메소드");
08    }
09 }
```

poly.sample.Child.java

```
01 package poly.sample;
02
03 public class Child extends Parent {
04    private int x=100;
05
06    public Child ( ) { }
07
08    public void out() {
09       System.out.println("부모의 protected num 필드 : " + num);
10       System.out.println("자식 클래스 메소드");
11    }
12
```

```
13      @Override
14      public void display() {
15          System.out.println("상속받아 재정의한 메소드");
16      }
17  }
```

test.poly.TestPolymorphism.java

```
01  package test.poly;
02
03  import poly.sample.*;
04
05  public class TestPolymorphism {
06      public static void main(String[] args) {
07          Child c = new Child();
08          c.display();
09          c.out();
10          System.out.println();
11
12          Parent p = new Child();
13          p.display();  //Overriding 된 parent 멤버이므로 접근 가능
14          //p.out();  //컴파일에러.  Parent 타입으로 Child 멤버에 접근할 수 없음.
15      }
16  }
```

```
          100
          상속받아 재정의한 메소드
 자식 클래스 메소드
          자식 클래스 메소드
```

Child c = new Child();	Parent p = new Child();
c.x → 접근 가능 c.display(); → 접근가능 c.out(); → 접근가능	p.x → 자식객체 멤버라 접근불가. p.display(); → 자식객체 오버라이딩된 메소드에 접근가능 p.out(); → 자식객체 멤버라 접근불가.

1.1 매개변수의 다형성

메소드의 매개변수에 다형성을 적용하면, 동일한 메소드의 오버로딩(overloading) 개수를 줄일 수 있게 된다. 메소드 매개변수 오버로딩의 한 예를 살펴보자.

test.poly.TestPolyArgument.java

```
01  package test.poly;
02
03  public class TestPolyArgument {
04          public static void main(String[] args) {
05                  Buyer  b = new Buyer();
06                  b.buy (new Chair( ));
07                  b.buy (new Desk( ));
08          }
09  }
10
11  class Furniture {
12          private int price; //제품가격
13          public Furniture (int price) {
14                  this.price = price;
15          }
16  }
17
18  class Chair extends Furniture {
19          public Chair() {
20                  super(100); // 부모 클래스의 생성자 호출
21          }
22          @Override
23          public String toString(){
24                  return "Chair";
25          }
26  }
27
28  class Desk extends Furniture {
29          public Desk() {
30                  super(200);
31          }
32
33          @Override
34          public String toString() {
35                  return "Desk";
36          }
37  }
38
```

```
39    class Buyer {
40        private int money = 500;
41
42        public void buy(Furniture f){
43            if (money<f.price) {
44                System.out.println("잔액부족!");
45                return;
46            }
47            money -= f.price;
48            System.out.println(f+" 구매성공! 잔액: "+money+" 만원");
49        }
50    }
```

```
Chair 구매성공! 잔액:400 원
Desk 구매성공! 잔액:200 원
```

예제의 클래스 Buyer 의 buy 메소드에 주목해 보자. 매개변수의 타입으로 Furniture 인 변수 f를 받고있다. 부모타입인 Furniture 덕분에 자식클래스인 Chair, Desk 모두 하나의 메소드를 사용할 수 있다. 만약 Chair, Desk 를 매개변수로 받았다면, 아래와 같이 각각 사용할 수 있는 메소드가 필요했을 것이다.

```
01  class Buyer {
02      int money = 500;
03      //Chair 구매메소드
04      void buy(Chair c){
05          if (money<c.price) {
06              System.out.println("잔액부족!");
07              return;
08          }
09          money -= c.price;
10          System.out.println(c+" 구매성공! 잔액: "+money+" 만원");
11      }
12      //Desk 구매메소드
13      void buy(Desk d){
14          if (money<d.price) {
15              System.out.println("잔액부족!");
16              return;
17          }
18          money -= d.price;
19          System.out.println(d+" 구매성공! 잔액: "+money+" 만원");
20      }
21  }
```

1.2 instanceof 연산자

매개변수의 타입으로 부모클래스를 선언해서 자식클래스를 모두 사용할 수 있는 클래스에서 각클래스마다 다르게 동작하는 기능이 필요하다면, 각 객체의 실제 타입을 알 수 있는 instanceof 연산자를 사용할 수 있다. 주로 조건식 안에서 사용되며, boolean값을 리턴하는 연산자 instanceof의 문법은 다음과 같다.

```
if (참조변수 instanceof 검사할 타입명(클래스명)) { }
```

instanceof의 연산 결과가 true 이면, 검사한 타입으로 형변환이 가능하다는 의미이다.

test.instance.TestInstanceOf.java

```
01  package test.instance;
02
03  public class TestInstanceOf {
04          public static void main(String[] args) {
05                  Chair c = new Chair();
06
07                  if (c instanceof Chair) {
08                          System.out.println("Chair 객체입니다.");
09                  }
10
11                  if (c instanceof Furniture) {
12                          System.out.println("Furniture 객체입니다.");
13                  }
14
15                  if (c instanceof Object) {
16                          System.out.println("Object 객체입니다.");
17                  }
18
19                  System.out.println("실제 타입 : "+ c.getClass().getName());
20          }
21   }
22
23   class Furniture { }
24
25   class Chair extends Furniture { }
```

```
Chair 객체입니다.
Furniture 객체입니다.
Object 객체입니다.
실제타입 : Chair
```

생성된 instance는 Chair 타입이지만, Object타입와 Furniture타입의 instanceof 연산결과로 모두 true값을 얻었다. Chair는 Furniture클래스를 상속하고 있지만, 모든 클래스는 또한 Object클래스의 자식클래스이기 때문이다. 또 검사한 타입(부모클래스)으로 형변환 할 수 있다는 의미이다.

.getClass().getName());는 참조변수가 가리키는 인스턴스의 클래스 이름을 문자열로 반환하는 Object클래스의 메소드이다. 모든 클래스는 Object 클래스를 상속하고 있기 때문에 이 메소드를 사용할 수 있다.

매개변수의 타입으로 부모클래스를 받아서, instanceof 연산을 통해 어느 후손 클래스 타입인지를 검사 후, 실제 클래스 타입으로 형변환할 수 있다. 이 때, 명시적으로 캐스팅해 줘야 한다. 캐스팅 후에는 부모 클래스 타입으로 사용할 수 없었던 자식 클래스 타입(객체의 실제타입)별 메소드를 사용할 수 있게 된다.

```
01  class Buyer {
02      int money = 500;
03      void buy(Furniture f){
04          if (f instanceof Chair) {
05              Chair c = (Chair)f;
06              c.chairMethod(); //Chair클래스에 선언된 메소드
07          }
08          else if (f instanceof Desk) {
09              Desk d = (Desk)f;
10              d.deskMethod(); //Desk클래스에 선언된 메소드
11          }
12
13  }
```

2 추상 (Abstract) 클래스

추상 클래스는 미완성된 클래스를 뜻하며, 선언되어 있지만 구현되지 않은 미완성의 메소드(추상 메소드)를 포함하고 있는 클래스일 때 반드시 astract 키워드를 class 앞에 붙여 준다.
클래스로서의 객체 생성은 못 하지만 새로운 클래스를 작성할 때 부모 클래스로 이용되며 새 클래스 작성을 위한 템플릿이 되어 상속 받은 클래스들의 규칙을 주는 용도로 사용 가능하다.

```
public abstract class 클래스이름 {
  // 필드, 생성자, 메소드 작성
  // 미완성된 추상 메소드 선언
}
```

클래스 앞에 'abstract' 키워드를 붙이면 추상 클래스가 생성되며, 객체를 생성할 수 없다는 점만 빼면 일반 클래스와 다른 점이 없다.

2.1 추상 (abstract) 메소드

메소드는 접근제어자, 리턴타입, 이름, 매개변수로 이루어진 선언부와 메소드의 기능을 정의하는 구현부로 나뉜다. 추상 메소드는 구현부('{}' Body) 를 가지지 않는다.

```
접근제한자 abstract 리턴타입 함수이름 ( );
```

추상 클래스에서 추상 메소드를 제공하는 이유는, 추상 클래스를 상속받아 만들어지는 후손 클래스들이 메소드를 처음 선언과 동일하게 재정의해서 사용하라는 의미이다. 반드시 구현부를 작성해야 하므로 강제성도 가지고 있다.

2.2 추상 클래스 작성

inherit.sample.Person.java

```
01  package inherit.sample;
02
03  public abstract class Person {
04      private String name;
05
06      public Person( ) { }
07      public void setName (String name) {   this.name = name;    }
08      public String getName ( ) {    return name;    }
09      public abstract void out ( );
10  }
```

Student.java

```
01  package inherit.sample;
02
03  public class Student extends Person{
04     private String subject;
05
06     public Student ( ) {    super ( );    }
07     public Student (String subject) {
08        super ( );
09        this.subject = subject;
10     }
11
12     public Student (String name, String subject) {
13        super(name);
14        this.subject = subject;
15     }
16
17     @Override
18     public void out ( ) {  //상속받은 부모의 추상메소드는 반드시 오버라이딩해야 함
19        System.out.println(getName ( ) + " 학생입니다");
20        System.out.println(subject + "를 수강합니다");
21     }
22  }
```

Professor.java

```
01  package inherit.sample;
02
03  public class Professor extends Person {
04     private String major;
05
06     public Professor ( ) {    super ( );    }
07
08     public Professor (String major) {
09        super ( );
10        this.major = major;
11     }
12
13     public Professor (String name, String major) {
14        super (name);
```

```
15        this.major = major;
16    }
17
18    @Override
19    public void out() {
20        System.out.println(getName ( ) + " 교수입니다");
21        System.out.println(major + "를 전공합니다");
22    };
23 }
```

inherit.test.AbstractEx.java

```
01 package inherit.test;
02
03 public class AbstractEx {
04    public static void main(String[] args) {
05        Student stu = new Student("홍길동", "자바");
06        Professor prof = new Professor("김춘추", "컴퓨터과학");
07
08        stu.out ( );
09        prof.out ( );
10    }
11 }
```

```
홍길동 학생입니다
자바를 수강합니다
김춘추 교수입니다
컴퓨터과학를 전공합니다
```

3 인터페이스 (interface)

추상메소드로만 구성된 추상클래스를 인터페이스(interface) 라고 구분하였으며, 추상 클래스보다 추상화 정도를 높여 일반 메소드와 멤버변수를 멤버로 가질 수 없게 하였다.

추가로 상수(public static final 필드)만 멤버로 가진다.

인터페이스는 다중 상속에 제한이 없기 때문에 여러 추상화 메소드를 인터페이스로 작성함으로써 코드의 강제성을 부여할 수 있고, 여러 클래스에서 사용되는 동일한 상수일 경우 인터페이스에 정의한 후 해당 상수를 공유할 수 있게 하면 중복을 줄일 수 있다.

3.1 인터페이스 작성

추상 메소드를 작성하려면 abstract 키워드를 표시해야 하지만 인터페이스 내부에는 모든 메소드가 추상 메소드이므로 public abstract 키워드를 생략할 수 있다. 하지만 혼동할 수 있으므로 모든 인터페이스의 추상 메소드에 public abstract 를 표시해 두기도 한다.

```
public interface 인터페이스이름 {
  // 상수 또는 추상 메소드
}
```

3.2 인터페이스 상속

클래스의 상속은 단일 상속만 가능하지만, 인터페이스는 다중 상속이 가능하다.

```
public interface 인터페이스 extends 인터페이스1, 인터페이스2, ... {
  // 상수 또는 추상 메소드
}
```

추상 메소드에 대한 구현 (implement) 과 재정의 (Overriding)는 인터페이스를 상속받은 후손 클래스에서 구현(Implement)해야 한다.

3.3 인터페이스 구현 (implement)

인터페이스가 가진 추상 메소드를 구현하려면 'implements' 키워드를 사용해서 후손 클래스가인터페이스를 상속을 받아야 한다. 상속받은 후손 클래스는 반드시 인터페이스의 추상 메소드를 구현(implement : 메소드 바디 추가하고 코드 작성)해야 한다.

```
public class 클래스이름 implements 인터페이스 {
  // 일반 클래스 정의
  // 인터페이스 추상 메소드 구현
}
```

3.4 인터페이스 장점

인터페이스를 이용하면 정의해야 하는 메소드(프로그램기능)를 표준화하고 강제화할 수 있다. 또한 메소드화 시켜야 하는 기능을 분류해야 하는 고민없이 구현만 하면 되므로 개발시간을 단축시킬 수 있다. 일반 클래스 상속을 이용해서 자식 클래스들의 관계를 맺는 것보다 간편하게 관계를 맺을 수 있다.

Chapter 12. 공공데이터 객체 변환처리하기

1. 공공데이터 제공 사이트 접속하기

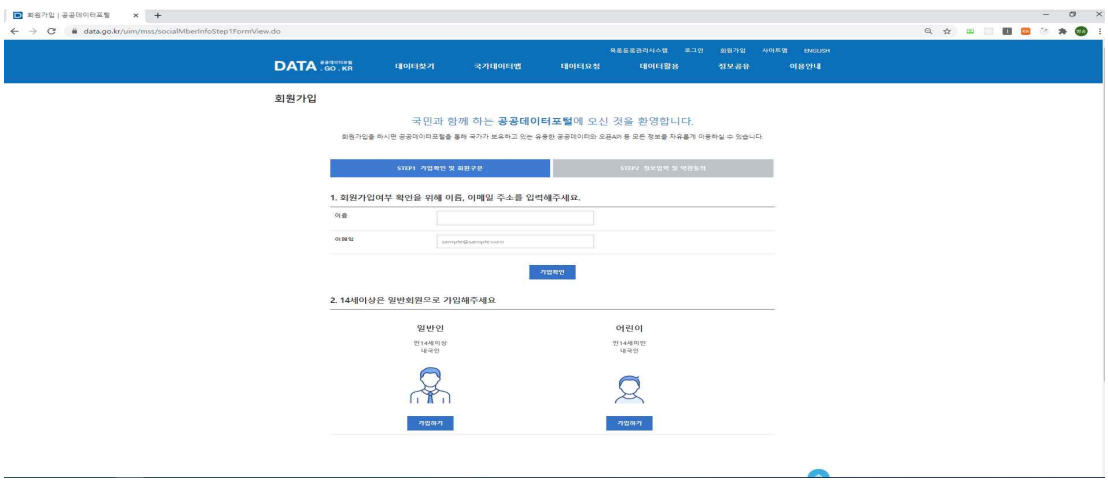
정부에서 제공하는 공공데이터 포털사이트를 이용하면 공공데이터를 손쉬게 이용할 수 있다. 브라우저를 열고 https://www.data.go.kr/index.do 접속하여 정부에서 제공하는 공공데이터를 활용할 수 있다.



중간에 있는 검색창을 이용해서 원하는 데이터를 검색할 수가 있고 세부검색 조건을 아래 있는 상세조건으로 검색할 수 있다.

2. 공공데이터 회원가입하기

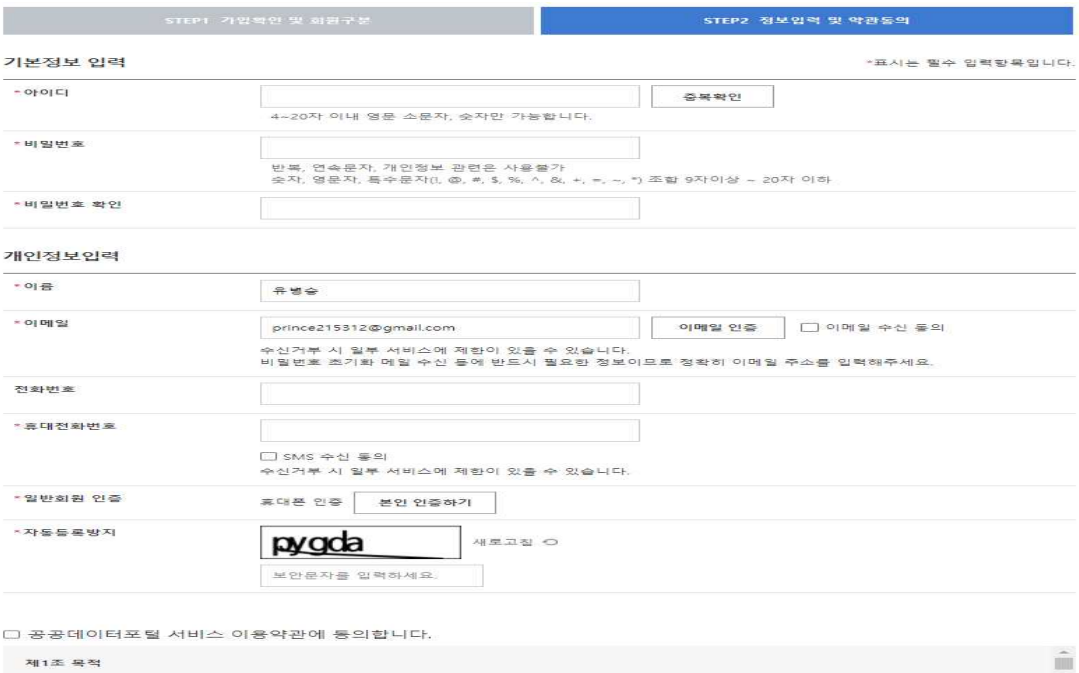
일단 공공데이터포털에서 제공하는 공공데이터를 이용하려면 회원가입을 해야한다.



1. 이름과 사용이 가능한 email 을 입력하고 가입확인을 한다. -> 가입정보가 없음 출력

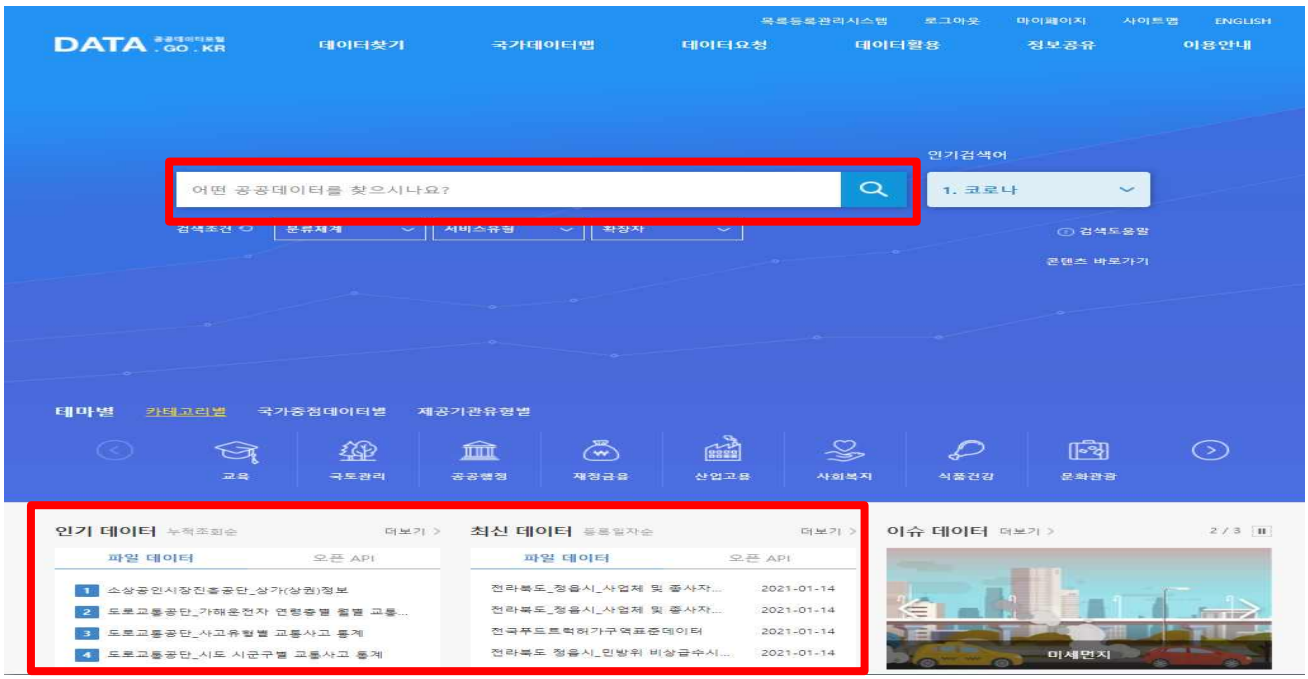
2. 연력에 맞춰 가입하기를 선택한다.

3. 각 항목에 상세 정보를 입력하여 회원가입을 완료한다.



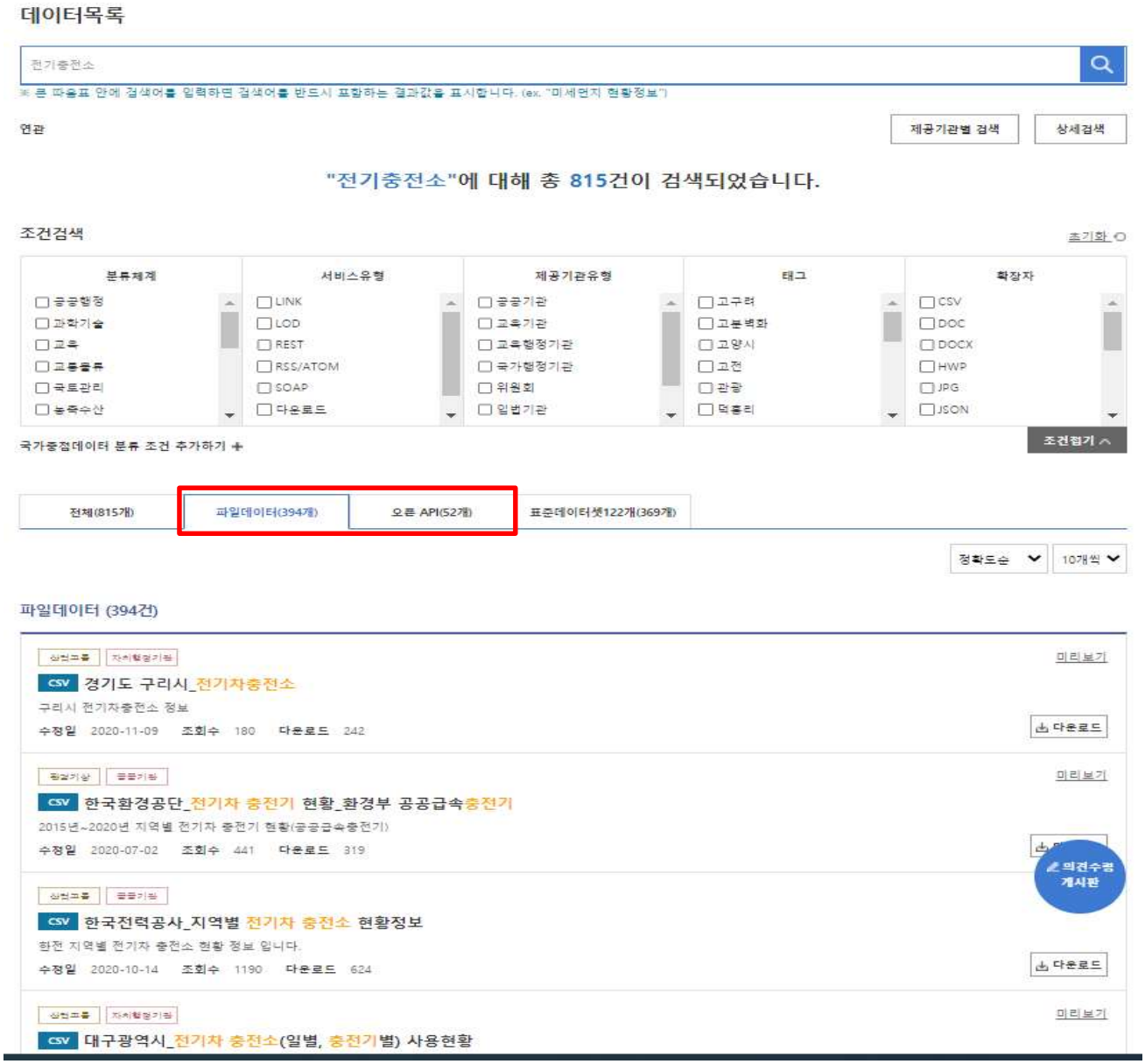
3. 공공데이터 조회하기

필요한 데이터는 공공데이터 포털 사이트의 메인화면 조회창을 이용해서 검색할수도 있고 인기데이터, 최신데이터 등 링크로 걸려있는 추천 데이터를 확인할 수 있다.



3.1 조회창 이용해서 데이터 검색하기

원하는 데이터 내용을 조회창에 검색을 하면 검색 결과가 나오고 검색결과는 파일데이터, 오픈 API 로 구분하여 확인할 수 있다. 파일데이터는 데이터가 파일로 저장되어있는 형태를 말하는것이고 오픈 API 는 요청주소로 HttpRequest 를 보내서 받아오는 방식을 의미한다. 검색 조건을 상세화하여 필터링도 가능하다.



3.2 파일데이터 상세보기

파일데이터 상세보기를 하면 데이터가 파일을 다운로드할 수 있는 버튼이 있고, 데이터 관련 상세내용을 확인할 수 있다. 미리보기를 보면 데이터의 구조와 내용을 미리 확인할 수 있다. 파일로 제공되는 공공데이터는 별로 인증키 없이 다운로드가 가능하다

파일데이터 상세

URL 복사

CSV 경기도 구리시_전기차충전소

구리시 전기차충전소 정보

0 0 관심

다운로드

오류신고 및 담당자 문의

파일데이터 정보

메타데이터 다운로드

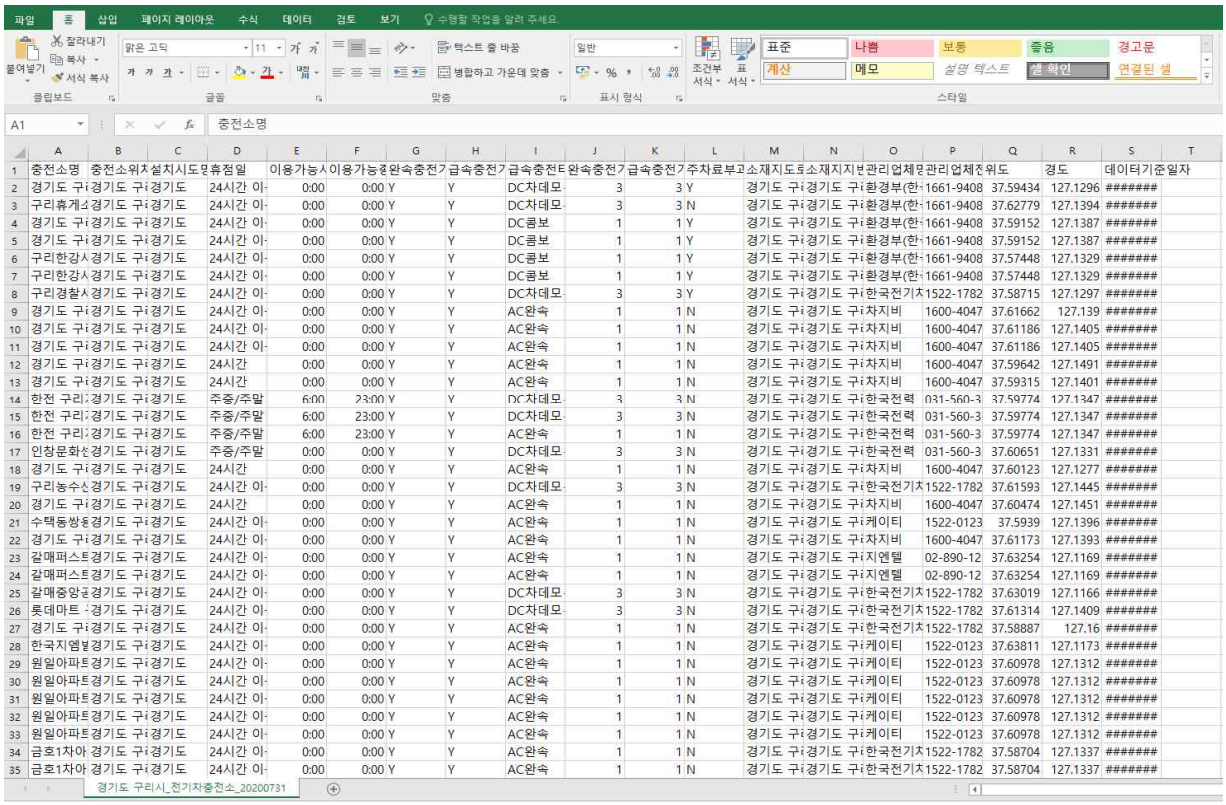
파일데이터명	경기도 구리시_전기차충전소_20200731		
분류체계	산업·통상·중소기업 - 에너지및자원개발	제공기관	경기도 구리시
관리부서명	정보통신과	관리부서 전화번호	031-550-2087
보유근거		수집방법	
업데이트 주기	연간	차기 등록 예정일	2021-07-31
매체유형	텍스트	전체 행	44
확장자	CSV	다운로드(바로가기)	244
데이터 한계		키워드	전기차,전기차충전,구리시
등록	2020-11-09	수정	2020-11-09
제공형태	공공데이터포털에서 다운로드(원문파일등록)		
설명	구리시 전기차충전소 정보		
기타 유의사항			
비용부과유무	무료	비용부과기준 및 단위	건
이용허락범위	이용허락범위 제한 없음		

미리보기

※ 파일 데이터의 일부 내용을 제공하고 있으며, 전체 내용이 필요한 경우 해당 파일을 다운로드 받으시기 바랍니다.

충전소명	충전소위치상세	설치시도명	휴점일	이용가능시작시각	이용가능종료시각	완속충전가능여부	급속충전가능여부	급속충전타입구분	완속충전기대수
경기도 구리시청	경기도 구리시 아차산로 439 본관 흡연실 옆	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC차데모+AC3상+DC콤보	3
구리휴게소(외측방향)	경기도 구리시 서울외곽순환고속도로 32 (사노동)	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC차데모+AC3상+DC콤보	3
경기도 구리시 행정복지센터	경기도 구리시 체육관로 74	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
경기도 구리시 행정복지센터	경기도 구리시 체육관로 74	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
구리한강시민공원(주차장)	경기도 구리시 코스모스길14번길 249	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
구리한강시민공원(주차장)	경기도 구리시 코스모스길14번길 249	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC콤보	1
구리경찰서	경기도 구리시 아차산로 359	경기도	24시간 이용가능	00:00	00:00	Y	Y	DC차데모+AC3상+DC콤보	3

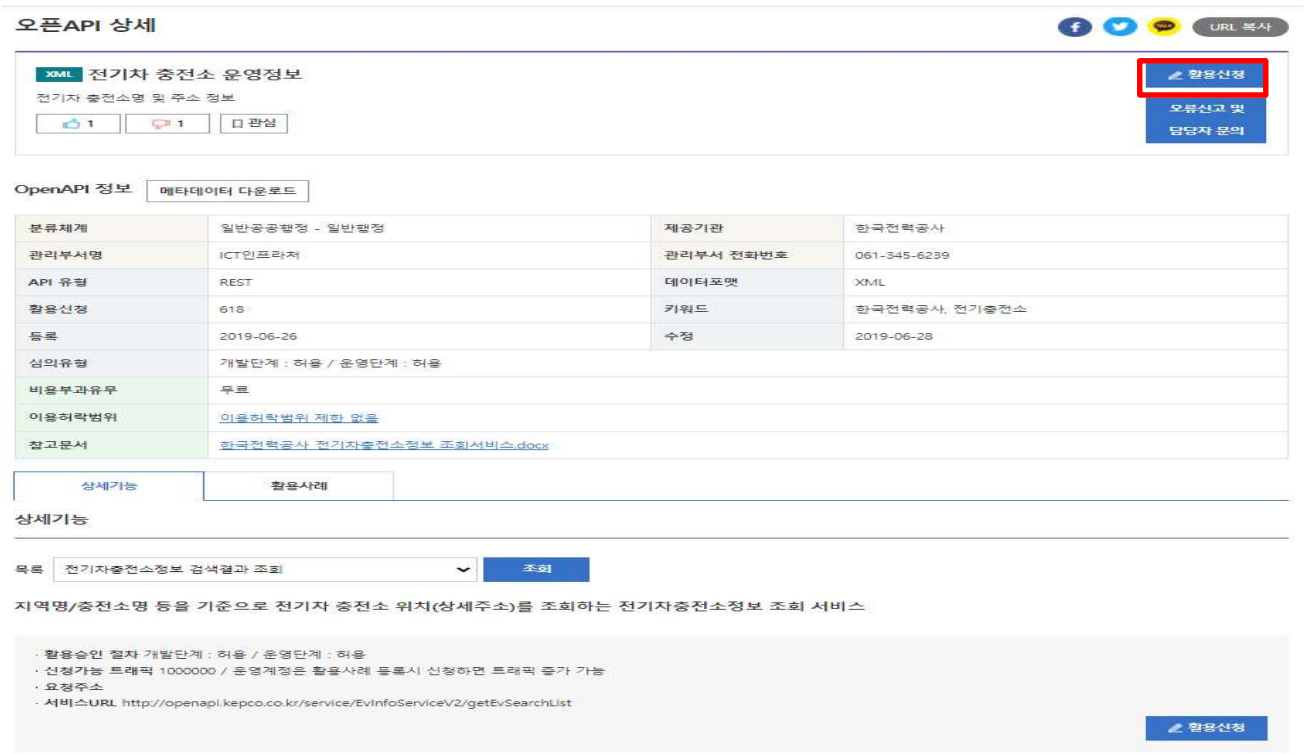
미리보기에 있는 데이터구조와 내용을 보고 데이터 저장방식 및 class 를 설계하면 된다. csv 파일을 다운로드 받으면 엑셀로 내부 데이터 전부를 확인할 수 있음.



충전소명	충전소위치	설치시도명	휴점일	이용가능시	이용가능종	완속충전기	급속충전기	급속충전타	완속충전기	급속충전기	주차료부과	소재지도로	소재지지번	관리업체명	관리업체전	위도	경도	데이터기준일자
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC차데모	3	3	Y	경기도 구리	경기도 구리	환경부(한국	1661-9408	37.59434	127.1296	########
구리휴게소	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC차데모	3	3	N	경기도 구리	경기도 구리	환경부(한국	1661-9408	37.62779	127.1394	########
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC콤보	1	1	Y	경기도 구리	경기도 구리	환경부(한국	1661-9408	37.59152	127.1387	########
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC콤보	1	1	Y	경기도 구리	경기도 구리	환경부(한국	1661-9408	37.59152	127.1387	########
구리한강시	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC콤보	1	1	Y	경기도 구리	경기도 구리	환경부(한국	1661-9408	37.57448	127.1329	########
구리한강시	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC콤보	1	1	Y	경기도 구리	경기도 구리	환경부(한국	1661-9408	37.57448	127.1329	########
구리경찰서	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC차데모	3	3	Y	경기도 구리	경기도 구리	한국전기차	1522-1782	37.58715	127.1297	########
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.61662	127.139	########
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.61186	127.1405	########
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.61186	127.1405	########
경기도 구리	경기도 구리	경기도	24시간	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.59642	127.1491	########
경기도 구리	경기도 구리	경기도	24시간	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.59315	127.1401	########
한전 구리지	경기도 구리	경기도	주중/주말	6:00	23:00	Y	Y	DC차데모	3	3	N	경기도 구리	경기도 구리	한국전력	031-560-3	37.59774	127.1347	########
한전 구리지	경기도 구리	경기도	주중/주말	6:00	23:00	Y	Y	DC차데모	3	3	N	경기도 구리	경기도 구리	한국전력	031-560-3	37.59774	127.1347	########
한전 구리지	경기도 구리	경기도	주중/주말	6:00	23:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	한국전력	031-560-3	37.59774	127.1347	########
인창문화소	경기도 구리	경기도	주중/주말	0:00	0:00	Y	Y	DC차데모	3	3	N	경기도 구리	경기도 구리	한국전력	031-560-3	37.60651	127.1331	########
경기도 구리	경기도 구리	경기도	24시간	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.60123	127.1277	########
구리농수산	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC차데모	3	3	N	경기도 구리	경기도 구리	한국전기차	1522-1782	37.61593	127.1445	########
경기도 구리	경기도 구리	경기도	24시간	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.60474	127.1451	########
수택동쌍용	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	케이티	1522-0123	37.5939	127.1396	########
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	차지비	1600-4047	37.61173	127.1393	########
갈매퍼스트	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	지엔텔	02-890-12	37.63254	127.1169	########
갈매퍼스트	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	지엔텔	02-890-12	37.63254	127.1169	########
갈매중앙로	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC차데모	3	3	N	경기도 구리	경기도 구리	한국전기차	1522-1782	37.63019	127.1166	########
롯데마트 구	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	DC차데모	3	3	N	경기도 구리	경기도 구리	한국전기차	1522-1782	37.61314	127.1409	########
경기도 구리	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	한국전기차	1522-1782	37.58887	127.16	########
한국지엠본	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	케이티	1522-0123	37.63811	127.1173	########
원일아파트	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	케이티	1522-0123	37.60978	127.1312	########
원일아파트	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	케이티	1522-0123	37.60978	127.1312	########
원일아파트	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	케이티	1522-0123	37.60978	127.1312	########
원일아파트	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	케이티	1522-0123	37.60978	127.1312	########
원일아파트	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	케이티	1522-0123	37.60978	127.1312	########
금호1차아	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	한국전기차	1522-1782	37.58704	127.1337	########
금호1차아	경기도 구리	경기도	24시간 이용	0:00	0:00	Y	Y	AC완속	1	1	N	경기도 구리	경기도 구리	한국전기차	1522-1782	37.58704	127.1337	########

3.3 오픈 API 상세보기

오픈 API 이용하는 요청 URL 주소와 요청시 파라미터정보, 응답하는 데이터에 대한 정보를 상세하게 보여준다. 오픈 API 활용에 대한 문서가 있어 이를 참고하면 손쉽게 이용할 수 있다. 그리고 오픈 API 를 통해서 공공데이터를 이용하려면 활용신청을 해서 인증키를 받아서 요청을 보내야한다.



요청변수(Request Parameter)

항목명(국문)	항목명(영문)	항목크기	항목구분	샘플데이터	항목설명
서비스키	ServiceKey	400	필수	공공데이터포털에서 받은 인증키	공공데이터포털에서 받은 인증키
페이지 번호	pageNo	4	필수	11	페이지번호
한 페이지 결과 수	numOfRows	4	필수	10	한 페이지 결과 수
충전소 주소	addr	255	옵션	전라남도 나주시 전력로 55	검색대상 충전소주소

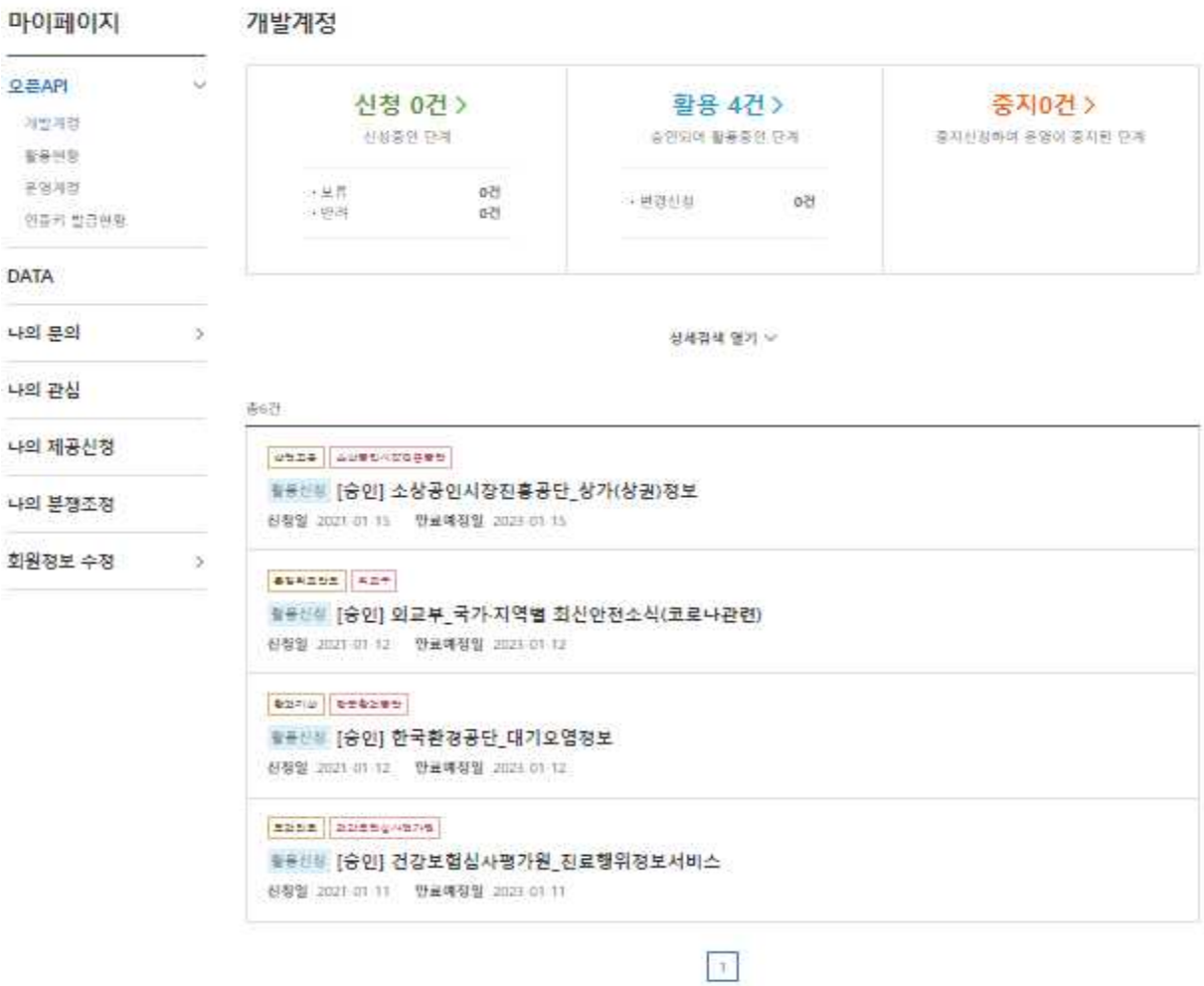
출력결과(Response Element)

항목명(국문)	항목명(영문)	항목크기	항목구분	샘플데이터	항목설명
결과코드	resultCode	2	필수		결과코드
결과메시지	resultMsg	50	필수		결과메시지
한 페이지 결과 수	numOfRows	4	필수	10	한 페이지 결과 수
페이지 번호	pageNo	4	필수	1	페이지번호
전체 결과 수	totalCount	4	필수	8	전체 결과 수
충전기주소	addr	255	필수	전라남도 나주시 전력로 55	충전소주소
충전기타입	chargeTp	1	필수	2	1 : 완속 2 : 급속
충전기ID	cpId	12	필수	13	충전기ID
충전기명칭	cpNm	20	필수	급속01	충전기명칭
충전기상태코드	cpStat	2	필수	1	1 : 충전가능 2: 충전중 3: 고장/점검 4 : 통신장애 5 : 통신미연결
충전방식	cpTp	2	필수	8	1 : B타입(5핀) 2 : C타입(5핀) 3 : BC타입(5핀) 4 : BC타입(7핀) 5 : DC차데모 6 : AC3상 7 : DC콤보 8 : DC차데모+DC콤보
충전소ID	csId	12	필수	0	충전소ID
충전소 명칭	csNm	50	필수	본사(나주)	충전소 명칭
위도	lat	20	필수	35.02636975	위도
경도	longi	20	필수	126.7844551	경도
충전기 상태 갱신 시각	statUpdatetime	20	필수	2017-02-22 13:02:22	충전기 상태 갱신 시각

요청변수는 공공데이터 서버에 데이터를 보낼 때 인수를 의미하고 출력결과는 응답한 데이터를 의미한다. 응답한 데이터를 기준으로 자바에서 데이터를 클래스를 설계하면 된다.



활용목적을 작성하고 활용신청을 하면 바로 인증키가 발행되고 1~2 시간 이후에 활용이 가능함.

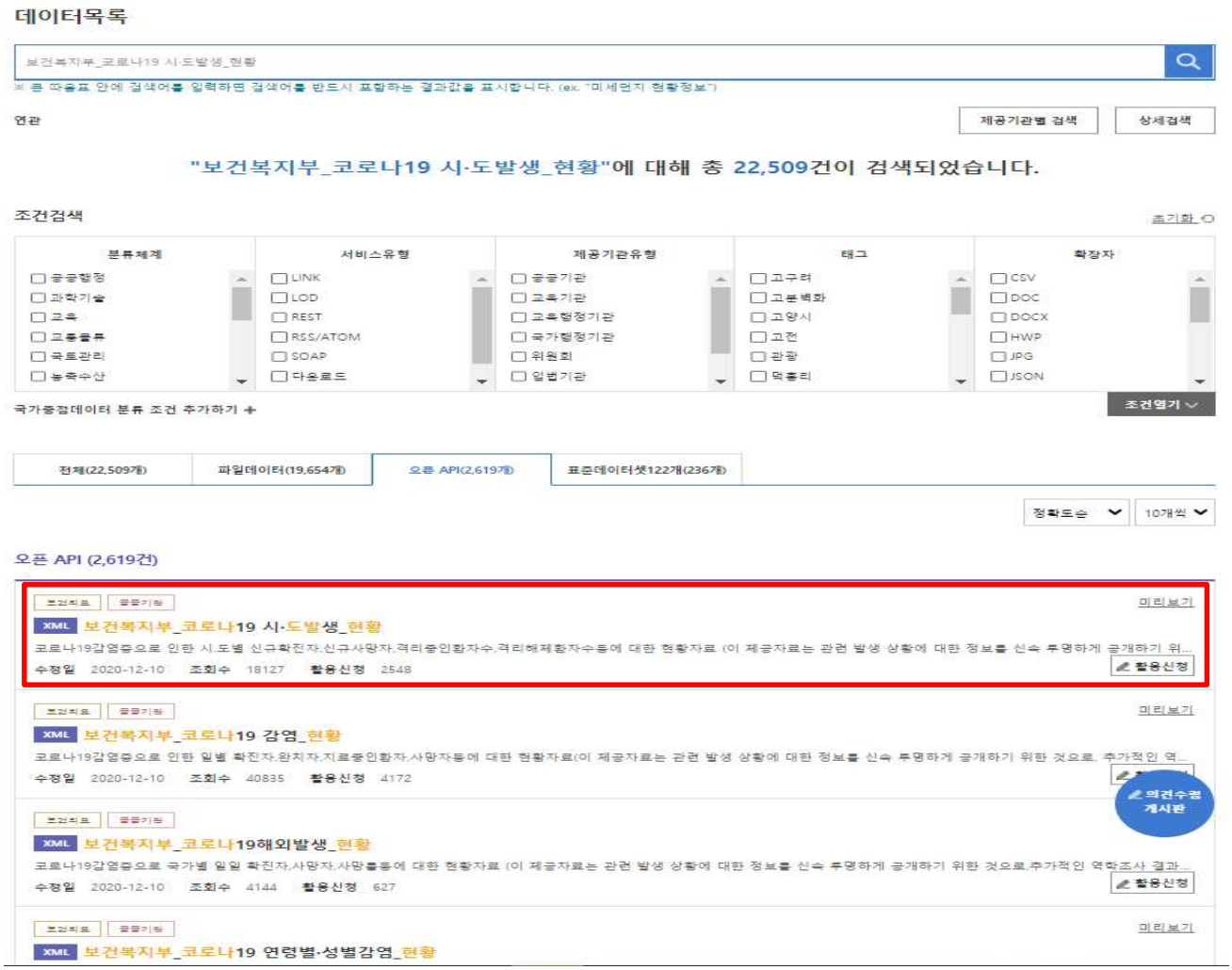


4 공공데이터 java클래스로 저장하기

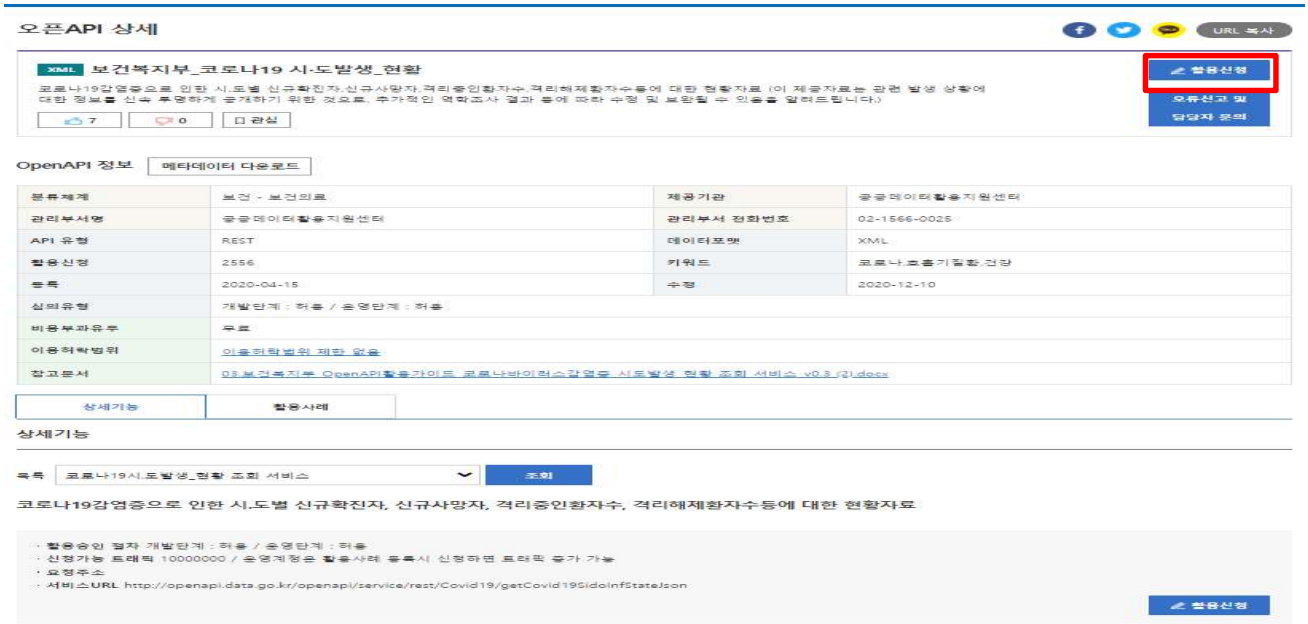
오픈API를 이용하여 공공데이터 활용하기 코로나19관련 데이터를 java클래스로 변환해 보도록 하겠다. openAPI를 이용해서 XML형식의 데이터를 가져와서 처리하겠다.

4.1 공공데이터 구조 분석 후 클래스 설계하기

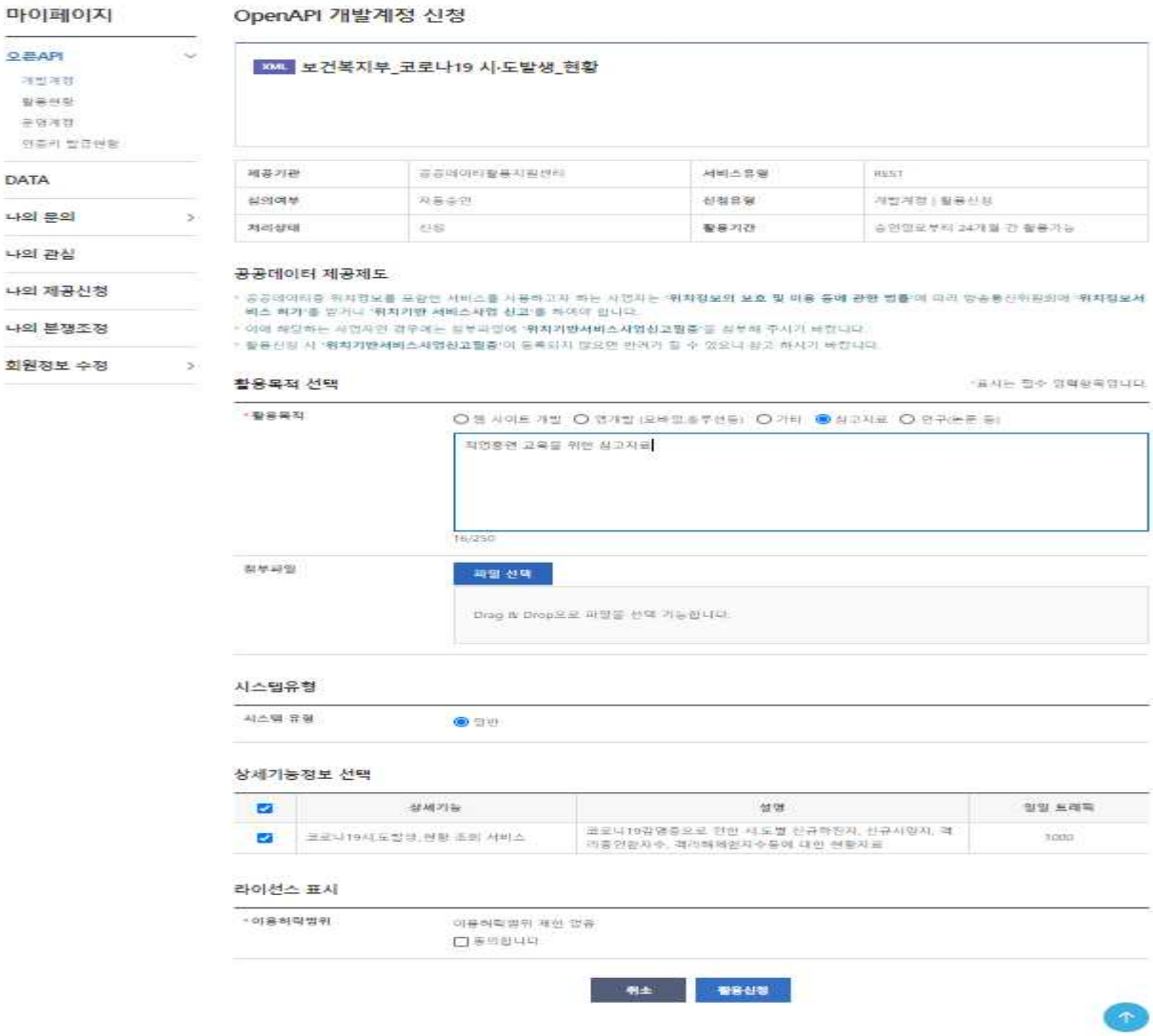
데이터 선정하기 공공데이터포털에 접속해서 "보건복지부_코로나19 시·도발생_현황"을 조회하면 첫번째 자료에 보건복지부_코로나19 시도발생_현황이 나오는데 그 자료를 샘플로 사용하도록하겠다.



상세화면으로 가서 오픈API를 사용해서 데이터를 이용하도록 하겠다. 오픈 API를 사용하기 위해서는 활용신청을 하여 인증키를 받아야지 사용이 가능하다. 상세화면에서 활용신청을 하고 인증키를 받도록 하자.

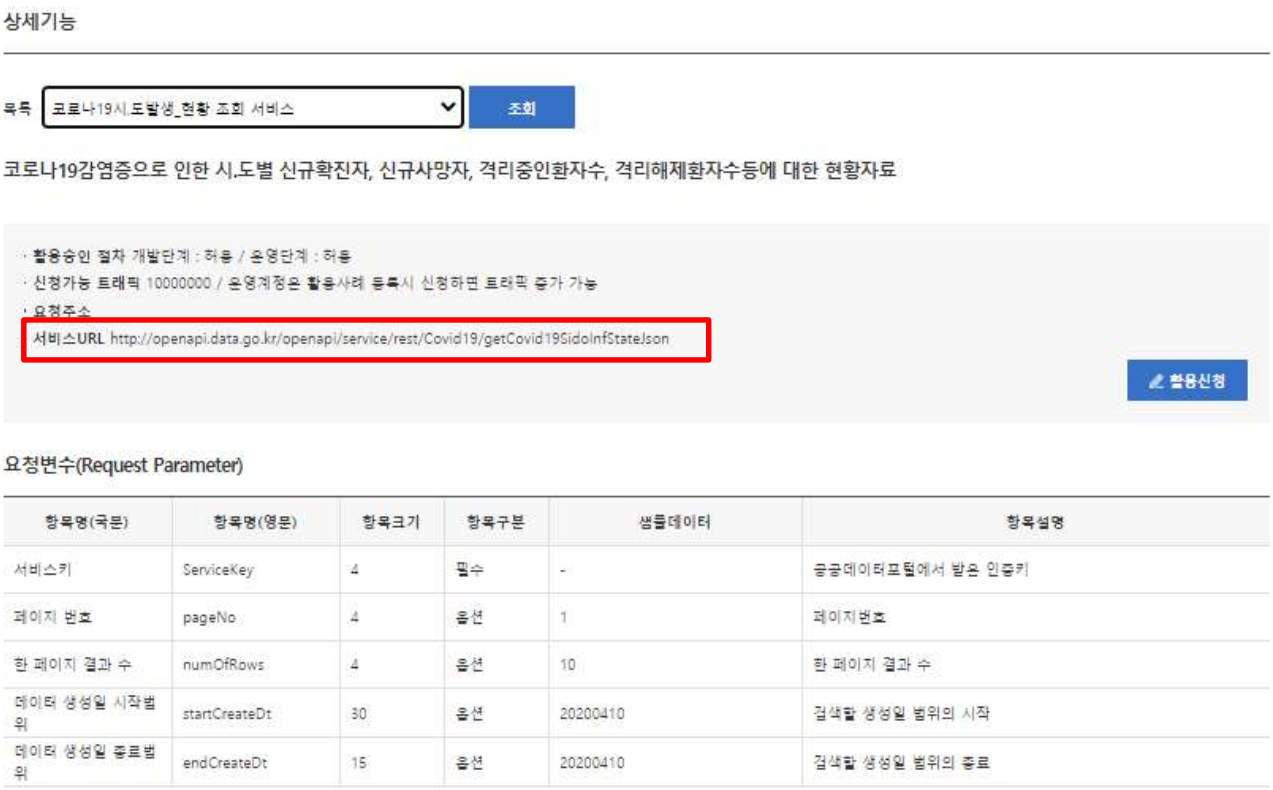


활용신청 버튼을 누르면 신청하는 화면이 나오고 활용목적에는 참고자료를 선택하고 직업훈련교육 참고자료로 작성, 통의합니다를 체크하고 활용신청을 하면 된다. 신청후에 1~2시간 혹은 24시간 이후에 사용이 가능하다는 안내가 나오고 신청이 완료됩니다. 1~2시간이 소요되기 때문에 개발전에 미리 활용신청을 해놓는 것이 좋다.



신청을 하고 마이페이지에 가보면 신청한 공공데이터가 있는 것을 볼수 있다.

오픈API를 사용하기 위해 신청한 공공데이터에 대한 정보를 살펴보자. 오픈API를 이용해서 데이터를 가져오는 방법은 요청(HttpRequest 요청)을 보내는 방식입니다. 요청을 보내면 서버에서 xml형식, json형식으로 데이터를 응답해 준다. 이 데이터를 받아서 java에서 활용할 수 있게 파싱처리하면 된다. 그러면 데이터를 어떻게 요청하고 어떻게 받는 지 알아야 한다. 그내용은 공공데이터 포털 상세화면에 자세하게 나오게 된다. 공공데이터포털 상세화면에서 참고문서에 정확하게 내용이 나오게 되니 꼭 참고해서 분석을 먼저해서 이용을 해야한다.



요청변수(Request Parameter)

항목명(국문)	항목명(영문)	항목크기	항목구분	샘플데이터	항목설명
서비스키	ServiceKey	4	필수	-	공공데이터포털에서 받은 인증키
페이지 번호	pageNo	4	옵션	1	페이지번호
한 페이지 결과 수	numOfRows	4	옵션	10	한 페이지 결과 수
데이터 생성일 시작범위	startCreateDt	30	옵션	20200410	검색할 생성일 범위의 시작
데이터 생성일 종료범위	endCreateDt	15	옵션	20200410	검색할 생성일 범위의 종료

상세내용을 보면 먼저 요청을 보낼 URL주소가 나오게 된다. 그 주소로 HttpRequest를 보내게 되면 원하는 시도별 코로나 감염자현황을 받을 수가 있다. 보낼때 서버에서 요청하는 데이터들이 있는데 그것을 요청변수라고 한다. 요청변수를 요청을 보내 데이터를 받을대 꼭 필요한 데이터기 때문에 반드시 알아야한다. 항목구분에 필수는 반드시 보내야하는 데이터이고 옵션은 선택적으로 보낼수있고 보내지 않으면 default값으로 설정이 된다.

요청주소는 서비스URL로 명시가 되어 있다

서비스 : http://openapi.data.go.kr/openapi/service/rest/Covid19/getCovid19SidoInfStateJson

요청변수와 함께 요청을 보내면 공공데이터에서 제공하는 데이터를 받아올 수 있다.

브라우저를 열고 주소창에

http://openapi.data.go.kr/openapi/service/rest/Covid19/getCovid19SidoInfStateJson?ServiceKey=xG7FbVRiB%2FVRULA5s7YlsBsZOsoP1a%2BS%2BdHFBLAaDX%2FRj3QgOTytvSMvI7EbGXFYK0yIDMpKM97N9%2F9awD9dYg%3D%3D

작성을 하면 응답데이터를 확인할 수 있다. 아래 사진과 같이 XML방식으로 데이터가 넘어오는 것을 확인할 수 있다.



위 사진에 나오는 데이터의 내용은 상세화면 출력결과부분에서 정확한 내용을 확인할 수 있다.

출력결과(Response Element)

항목명(국문)	항목명(영문)	항목크기	항목구분	샘플데이터	항목설명
결과코드	resultCode	2	필수	00	결과코드
결과메시지	resultMsg	50	필수	OK	결과메시지
한 페이지 결과 수	numOfRows	4	필수	10	한 페이지 결과 수
페이지 번호	pageNo	4	필수	1	페이지번호
전체 결과 수	totalCount	4	필수	3	전체 결과 수
게시글번호(국내 시도별 발생현황 고유값)	SEQ	30	필수	130	게시글번호(국내 시도별 발생현황 고유값)
등록일시분초	CREATE_DT	30		2020-04-10 11:15:59.026	등록일시분초
사망자 수	DEATH_CNT	15		204	사망자 수
시도명(한글)	GUBUN	30		부산	시도명(한글)
시도명(중국어)	GUBUN_CN	30		null	시도명(중국어)
시도명(영어)	gubunEn	30		null	시도명(영어)
전일대비 증감 수	INC_DEC	15		39	전일대비 증감 수
격리 해제 수	ISOL_CLEAR_CNT	15		6973	격리 해제 수
10만명당 발생률	QUR_RATE	30		20.10	10만명당 발생률
기준일시	STD_DAY	30		2020년 3월 13일 00시	기준일시
수정일시분초	UPDATE_DT	30		null	수정일시분초

이 xml형태로 넘어온 데이터를 분석하여 자바에서 활용할 수 있게 만들면 된다. xml형태는 태그형식으로 되어 있고, 기본구조는 아래와 같다.

root 태그 : response

자식태그

header : 해당서비스 코드, 메시지 정보

body : items 태그를 자식으로 가지고 items 실제 데이터이다.

numOfRows : 페이지당 데이터 수

pageNo : 페이지번호

totalCount : 전체 데이터수

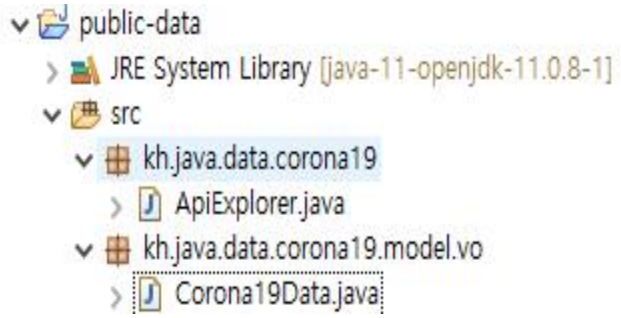
items태그의 자식태그로 item태그가 있고 iitem태그의 자식으로 사용할 데이터가 나오는데 출력결과의 항목명(영문)이 태그명이 된다. 데이터를 받아와 java 객체로 파싱할 때 필요하니 잘 기억을 해야한다.
출력결과의 샘플데이터를 보고 들어올 데이터의 타입을 예상할 수 있는데 거기에 맞춰 테이블을 설계할 수 있다. 항목명을 데이터로 분석해서 java클래스로 설계하면 다음과 같이 할 수 있다. 다른 공공데티어를 이용할때는 필요한 데이터만 java객체로 설정하여 이용할 수도 있다.



4.2 공공데이터 java객체로 변환하여 저장하기

프로젝트를 생성해서 공공데이터 요청 및 받아온 데이터 파싱처리해서 자바 애플리케이션에서 이용해보자.

프로젝트 구조



패키지를 두개로 분리하여 기능구현, 데이터 저장용 클래스를생성하여 실습을 하겠다.

ApiExplorer.java는 자바애플리케이션을 실행하는 클래스로 main메소드가 있고, 내부에 run(), parseXML()매소드를 가지고 있다.

run()매소드 : HttpUrlConnection을 이용하여 공공데이터서버에서 데이터를 가져와 데이터 저장용 클래스인 Corona19Data.java에 가져온 데이터를 넣는 기능을 한다.

parseXML()매소드 : Document객체를 이용해서 xml형식으로 넘어온 문자열 데이터를 document객체로 변환해서 각 태그에 접근하여 필요한 데이터를 가져올 수 있게 하는 기능을 한다.

Corona19Data.java는 가져온 데이터를 저장하는 클래스이다.

ApiExplorer.java 구현하기

Kh.java.data.corona19.ApiExplorer

```
package kh.java.data.corona19;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.UnsupportedEncodingException;
import java.net.HttpURLConnection;
import java.net.MalformedURLException;
import java.net.URL;
import java.net.URLEncoder;
import java.util.ArrayList;
import java.util.List;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;

import org.w3c.dom.Document;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

import kh.java.data.corona19.model.vo.Corona19Data;
```

```
public class ApiExplorer {
    public static void main(String[] args){
        new ApiExplorer().run();
    }


    private void run() {

        //1. 요청 url 생성
        StringBuilder urlBuilder = new
StringBuilder("http://openapi.data.go.kr/openapi/service/rest/Covid19/getCovid19SidoInfStateJson"); /*URL*/
        try {
                        urlBuilder.append("?" + URLEncoder.encode("ServiceKey","UTF-8")
+
"=xG7FbVRiB%2FVRULA5s7YlsBsZOsoP1a%2BS%2BdHFBLAa DX%2FRj3QgOTytvSMvI7EbGXFYK0yIDMpKM97N9%2F9awD9dYg%3D%3D");/*Service Key*/
                        urlBuilder.append("&" + URLEncoder.encode("pageNo","UTF-8") +
"=" + URLEncoder.encode("1", "UTF-8")); /*페이지번호*/
                        urlBuilder.append("&" + URLEncoder.encode("numOfRows","UTF-8") +
"=" + URLEncoder.encode("10", "UTF-8")); /*한 페이지 결과 수*/
                        urlBuilder.append("&" + URLEncoder.encode("startCreateDt","UTF-8") + "=" + URLEncoder.encode("20200410", "UTF-8")); /*검색할 생성일 범위의 시작*/
                        urlBuilder.append("&" + URLEncoder.encode("endCreateDt","UTF-8")
+ "=" + URLEncoder.encode("20200410", "UTF-8")); /*검색할 생성일 범위의 종료*/
                } catch (UnsupportedEncodingException e) {
                        e.printStackTrace();
                }

        //2. connection객체 생성
        URL url = null;
        HttpURLConnection conn = null;
                try {
                        url = new URL(urlBuilder.toString());
                        conn = (HttpURLConnection) url.openConnection();
                        conn.setRequestMethod("GET");
                        conn.setRequestProperty("Content-type", "application/json");
                        System.out.println("Response code: " + conn.getResponseCode());
                } catch (MalformedURLException e) {
                        e.printStackTrace();
                } catch (IOException e) {
                        // TODO Auto-generated catch block
                        e.printStackTrace();
                }

                //3. 요청전송 및 응답 처리
        BufferedReader br = null;
                try {
                        int statusCode = conn.getResponseCode();
                        if(statusCode >= 200 &&  statusCode <= 300)
                                br = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
                        else
                                br = new BufferedReader(new
InputStreamReader(conn.getErrorStream()));
```

```
                    //a.문자열로 확인
//                  StringBuilder sb = new StringBuilder();
//                  String line;
//                  while ((line = rd.readLine()) != null) {
//                          sb.append(line);
//                  }
//                  System.out.println(sb.toString());

                    //b.xml parsing
                    /*
                     <items>
                     <item>
                        <createDt>2020-04-10 11:17:34.589</createDt>
                        <deathCnt>0</deathCnt>
                        <defCnt>352</defCnt>
                        <gubun>검역</gubun>
                        <gubunCn>隔離區</gubunCn>
                        <gubunEn>Lazaretto</gubunEn>
                        <incDec>4</incDec>
                        <isolClearCnt>3</isolClearCnt>
                        <isolIngCnt>349</isolIngCnt>
                        <localOccCnt>0</localOccCnt>
                        <overFlowCnt>4</overFlowCnt>
                        <qurRate>-</qurRate>
                        <seq>1014</seq>
                        <stdDay>2020년 04월 10일 00시</stdDay>
                        <updateDt />
                     </item>
                            ...
                    </items>
                     */
                    Document doc = parseXML(conn.getInputStream());

                    //a. item 태그객체 목록으로 가져온다.
                    NodeList descNodes = doc.getElementsByTagName("item");

                    //b. Corona19Data List객체 생성
                List<Corona19Data> list = new ArrayList<>();

                    //c. 각 item태그의 자식태그에서 정보 가져오기
                for(int i = 0; i < descNodes.getLength(); i++){
                                //item
                    Node item = descNodes.item(i);
                    Corona19Data data = new Corona19Data();

                    //item 자식태그에 순차적으로 접근
                    for(Node node = item.getFirstChild(); node != null; node =
node.getNextSibling()){ //첫번째 자식을 시작으로 마지막까지 다음 형제를 실행
                            //System.out.println(node.getNodeName() + " : " +
node.getTextContent());
                            switch(node.getNodeName()) {
                            case "seq":
data.setSeq(Integer.parseInt(node.getTextContent())); break;
```

```
                                case "createDt": data.setCreateDt(node.getTextContent());
break;
                                case "deatCnt":
data.setDeathCnt(Integer.parseInt(node.getTextContent())); break;
                                case "defCnt":
data.setDefCnt(Integer.parseInt(node.getTextContent())); break;
                                case "gubun": data.setGubun(node.getTextContent());
break;
                                case "gubunCn": data.setGubunCn(node.getTextContent());
break;
                                case "gubunEn": data.setGubunEn(node.getTextContent());
break;
                                case "incDec":
data.setIncDec(Integer.parseInt(node.getTextContent())); break;
                                case "isolClearCnt":
data.setIsolClearCnt(Integer.parseInt(node.getTextContent())); break;
                                case "isolIngCnt":
data.setIsolIngCnt(Integer.parseInt(node.getTextContent())); break;
                                case "localOccCnt":
data.setLocalOccCnt(Integer.parseInt(node.getTextContent())); break;
                                case "overFlowCnt":
data.setOverFlowCnt(Integer.parseInt(node.getTextContent())); break;
                                case "qurRate": data.setQurRate(node.getTextContent());
break;
                                case "stdDay": data.setStdDay(node.getTextContent());
break;
                                case "updateDt": data.setUpdateDt(node.getTextContent());
break;
                                }
                        }

                                //d. List객체에 추가
                        list.add(data);

                    }

                        //e.최종확인
                    for(Corona19Data d : list)
                        System.out.println(d);



                } catch (IOException e) {
                        e.printStackTrace();
                } finally {
                        try {
                                br.close();
                        } catch (Exception e) {
                                e.printStackTrace();
                        }
                }
        //4. 자원반환
        conn.disconnect();
        }


        private Document parseXML(InputStream stream) {
```

```
        DocumentBuilderFactory objDocumentBuilderFactory =
DocumentBuilderFactory.newInstance();
        DocumentBuilder objDocumentBuilder = null;
        Document doc = null;

        try {
                        objDocumentBuilder =
objDocumentBuilderFactory.newDocumentBuilder();
                        doc = objDocumentBuilder.parse(stream);
                } catch (ParserConfigurationException e) {
                                e.printStackTrace();
                } catch (SAXException e) { //Simple API for XML
                        e.printStackTrace();
                } catch (IOException e) {
                        e.printStackTrace();
                }

        return doc;
    }
}
```

HttpURLConnection객체는 java코드로 HttpRequest를 보내고 응답을 받아올 수 있는 객체로 브라우저에 주소창에 URL창에 주소를 적어서 요청을 보내고 응답을 받는거와 동일한다. java에서 주소는 URL객체를 이용해서 작성을 한다. HttpURLConnection객체응 이용하기 위해서는 URL객체를 생성하고 openConnection()매소드를 이용하면 URLConnction객체를 반환하는데 이를 web전용 HttpURLConnection 객체로 형변환하여 이용하면 되겠다. openConnection매소드를 실행하면 요청을 보내고 응답은 getInputStream()매소드를 이용해서 전송되는 데이터를 가져올 수 있다. HttpURLConnection을 이용해서 요청방식(METHOD)이나 응답데이터형식(Content-type)을 정할 수 있다.

전송되는 데이터는 문자열타입으로 xml형태로 전송이 된다.

전송된 xml형태의 문자열 타입을 파싱하여 사용해야한다. 이때 com.w3c.dom.Document객체를 이용할 수 있다. xml형태는 태그방식의 데이터전송 전용 형식으로 web표준을 관리하는 w3c에서 관리하고 있다. Document객체를 이용하여 태그형식으로 되어 있는 내용에 접근하여 실제 필요한 데이터를 가져올 수 있다. 먼저 DocumentBuilder객체의 parse()매소드를 이용해서 문자열을 Document(xml형태)로 저장을 하고 Document객체에서 `getElementsByTagName("태그명")`매소드를 이용해서 태그가 가지고 있는 데이터를 가져올 수 있다.

실행결과확인



공공데이터에서 데이터를 가져와 자바객체에 저장하여 활용하는 방법에 대해 알아보았다. 공공데이터는 기관에서 지속적으로 제공하고 있어 이를 활용하면 더 유용한 정보를 제공하는 애플리케이션을 만드는데 많은 도움이 될 수 있다.